東北大學 国家示范性软件学院
SOFTWARE COLLEGE OF NORTHEASTERN UNIVERSITY



Digital Design and Computer Architecture

NEU-SWC-A0801040040

Chapter 1： From Zero to One 1

考核方式及成绩评定

总分100分，三部分组成

1. 线上成绩：MOOC学习记录，20%（包括：做周测题、做单元测验题、讨论区发声、完成线上期末试题）
2. 线下成绩：慕课堂10%（翻转课堂课内练习、主题讨论等）
3. 实验成绩：20%
4. 期末考试：50%，闭卷

雨课堂 Rain Classroom

线上线下结合的学习方式

- 登录中国大学MOOC网，查找计算机组成原理-东北大学



中国大学MOOC



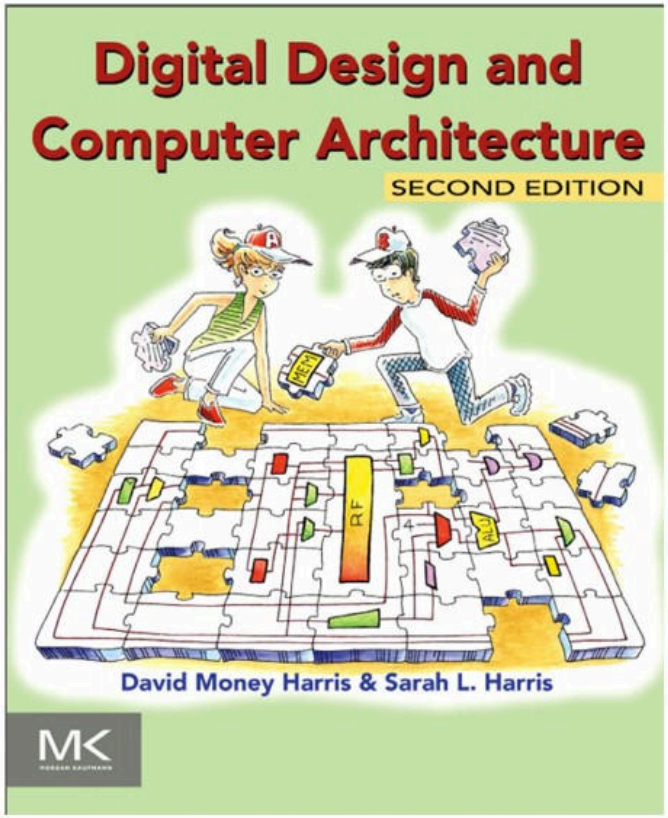
NEU-SWC-A0801040040 Chapter 1： From Zero to One 4

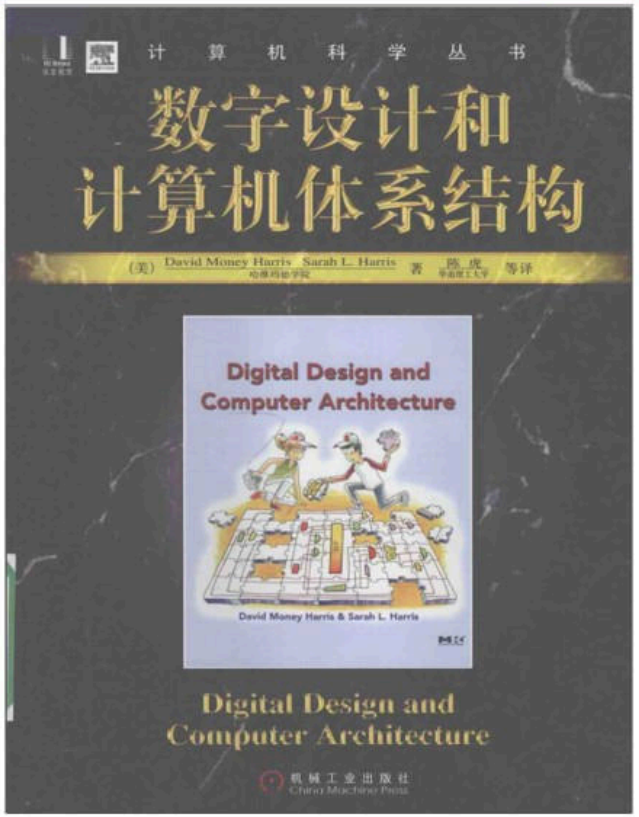
线下课程互动平台“慕课堂”

- 注意：MOOC 和 慕课堂，不一样
  - 线上学习使用MOOC（自己安排时间）
    - 看视频自学，线上练习，线上讨论，线上期末测验
  - 线下学习使用 “慕课堂” （随着理论课时间）
    - 上课签到，上课做练习，上课讨论等；
    - 第一次上课加入各自班级的慕课堂
- “慕课堂”账户要与“MOOC”一致，否则学习数据无法一起统计；

雨课堂 Rain Classroom

教材


Digital Design and
Computer Architecture
SECOND EDITION
David Money Harris & Sarah L. Harris


计 算 机 科 学 丛 书
数字设计和
计算机体系结构
Digital Design and
Computer Architecture
Digital Design and
Computer Architecture

参考书





雨课堂 Rain Classroom

单选题 1分

- 请看下面几段实现矩阵乘法C=A*B的代码，其中矩阵 A、B、C 均为 n 阶方阵，且 n 的数值很大。
- 哪段代码执行速度最快？还是都一样，还是b，c比a快？

```
//代码A
for( i = 0; i < n; i++) {
   for( j = 0; j < n; j++) {
      sum = 0.0;
      for(k = 0; k < n; k++)
         sum += a[i][k]*b[k][j];
      c[i][j] = sum;
   }
}
```

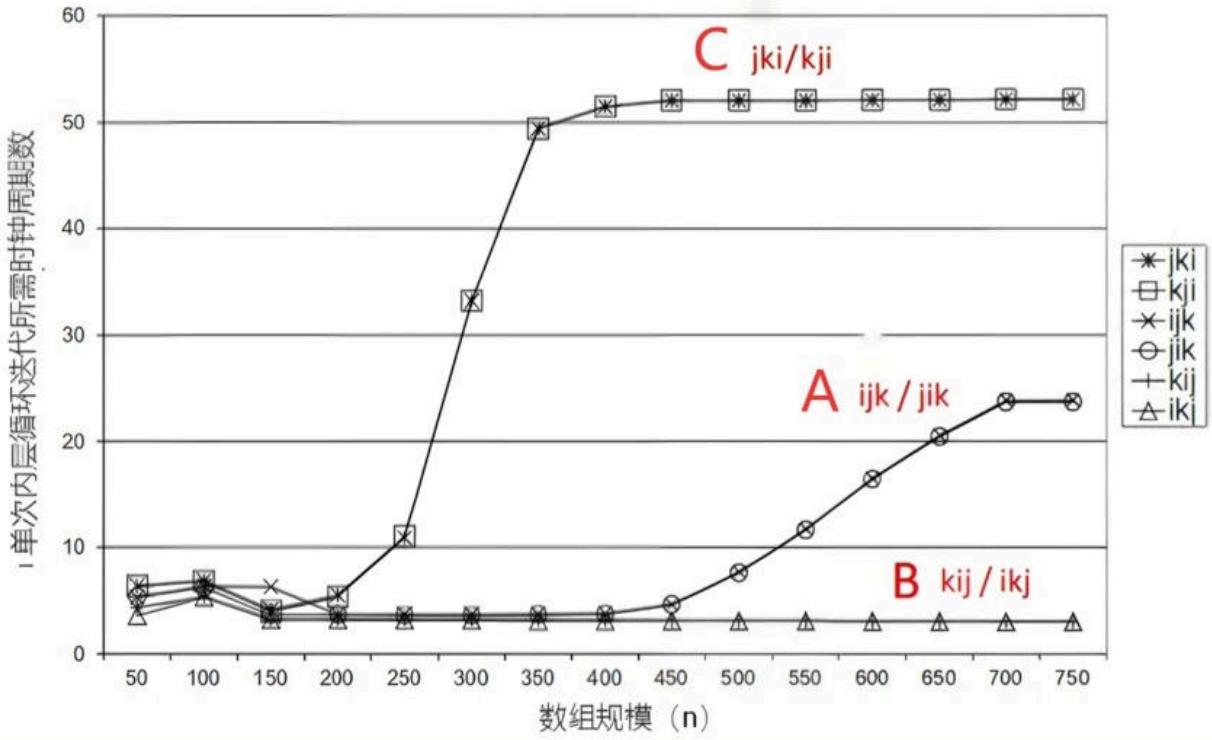
```
//代码B
for( k = 0; k < n; k++) {
   for( i = 0; i < n; i++) {
      r = a[i][k];
      for(j = 0; j < n; j++)
         c[i][j] += r*b[k][j];
   }
}
```

```
//代码C
for( j = 0; j < n; j++) {
   for( k = 0; k < n; k++) {
      r = b[k][j];
      for( i = 0; i < n; i++)
         c[i][j] += a[i][kj]*r;
   }
}
```

A 代码A　B 代码B　C 代码C　D 代码A/B/C一样快　E 代码B/C比A快

为什么学习这门课?

- Core i7 矩阵乘法的性能



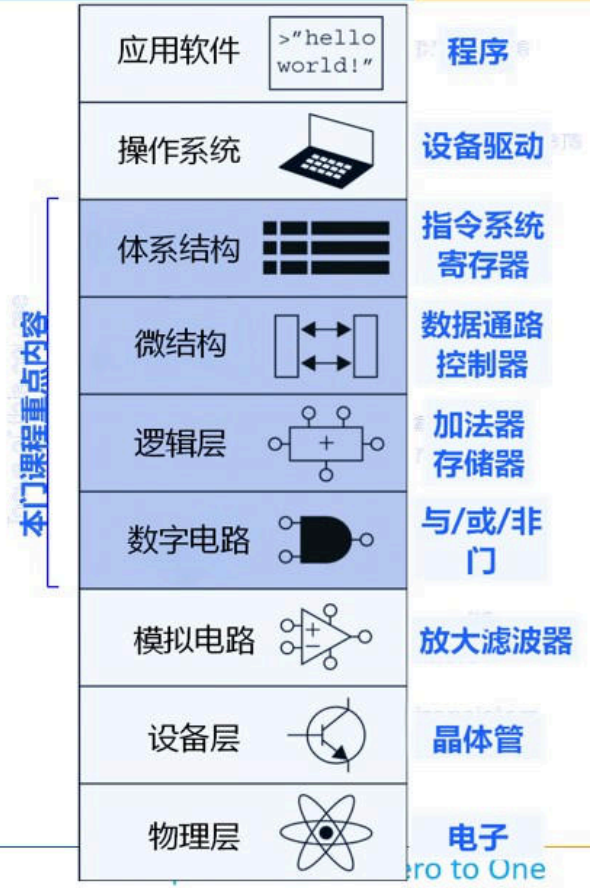
课程计划

- 课程目标:
  - 知道-了解-理解，电脑神秘面纱下有什么
  - 学习的数字设计的原则、原理
  - 设计和建造一个微处理器（CPU）

雨课堂 Rain Classroom

学习计划

- 计算机体系结构
  - 复杂系统：由2到4级功能部件组成
- 计算机组成原理
  - 更复杂的逻辑功能单元：微处理器
  - 逻辑功能单元：加法器、计数器、乘法器
- 逻辑与数字系统
  - 逻辑功能单元：与门、或门、非门
- 模拟电子
  - 电子元件：晶体管、二极管、电阻、电容



学习计划

- 计算机体系结构（Computer Architecture）
  - 程序员所见到的计算机系统的属性、概念性的结构与功能特性
  - 表现为：指令系统、数据类型、寻址技术、I/O机理等。如，有无乘法指令。
- 计算机组成原理（Computer Organization）
  - 实现计算机体系结构所体现的属性
  - 表现为：具体指令的实现。如，如何实现乘法指令
- 逻辑与数字系统（Logical and Digital System）
  - 计算机组成的物理实现
  - 如，乘法器用什么电子器件实现

雨课堂 Rain Classroom

单选题 1分

关于**计算机体系结构**（Computer Architecture）与**计算机组成原理**（Computer Organization）的关系，下列描述正确的是?

A 体系结构关注如何用晶体管实现加法器，组成原理关注指令集是否支持乘法

B 体系结构定义程序员可见的属性（如指令系统、寻址方式），组成原理实现这些属性（如乘法指令的微操作序列）

C 体系结构决定使用CMOS还是TTL工艺，组成原理决定Cache是否采用写直达策略

D 二者等价，只是不同教材使用的术语差异

雨课堂 Rain Classroom



Chapter 1 From Zero to One

Chapter 1 :: Topics

- 驾驭复杂的艺术 (The Art of Managing Complexity)
- 数制系统 (Number Systems)
- 数字抽象 (The Digital Abstraction)
- 逻辑门 (Logic Gates)
- CMOS晶体管 (CMOS Transistors)
- 功耗 (Power Consumption)

雨课堂 Rain Classroom

驾驭复杂的艺术

- 抽象（Abstraction）
- 约束（Discipline）
- 三个原则（The Three –Y's）
  - 层次化（Hierarchy）
  - 模块化（Modularity）
  - 规整化（Regularity）

数字抽象（The Digital Abstraction）

- 大多数的物理量都是连续的模拟量
  - 电线上的电压值
  - 震动的频率
  - 物体的位置等
- 数字抽象只考虑离散量的子集
  - 离散化为十进制?
  - 离散化为二进制?
  - 离散化为十六进制?
  - 离散化为...进制?

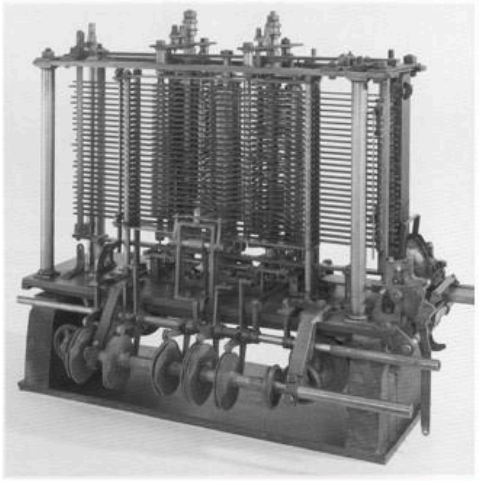


雨课堂 Rain Classroom

巴贝奇的分析机（课后自学）

- 从1834年-1871年巴贝奇Charles Babbage一直在设计和尝试制作这种机械计算机，
- 被公认为是首台数字计算机（现代电子计算机的前身）
- 由机械齿轮搭建而成，每一组齿轮代表一个离散数值（0 至 9），这很像汽车里面的机械里程表。
- 图中显示了这种分析机的原型，其中每一行表示一个数字，使用了25行齿轮，能表示25个数字
- 巴贝奇在该设备完工前便已离世





雨课堂 Rain Classroom

约束（Discipline）

- 拿软件开发举例：
  - 作为软件开发者，我们设计的每一个软件都存在于现实生活中，所以一定会存在约束条件。比如有时间和预算的约束，还有技术约束，你肯定不能接受一个你的技术达不到的项目，或者说某个功能你的技术达不到，你也不可能非要去实现。
- 对于硬件设计同样有约束
  - 采用离散电压(0v, 5v)，而非连续电压(0v-5v)
  - 相比模拟电路，数字电路设计难度更低，可搭建更复杂精密的系统
- 数字系统正逐步取代先前的模拟设备：
  - 例如数码相机、数字电视、手机等

雨课堂 Rain Classroom

驾驭复杂的艺术

- 除了前面提到的抽象和约束外，设计者还使用三条原则来处理系统的复杂性

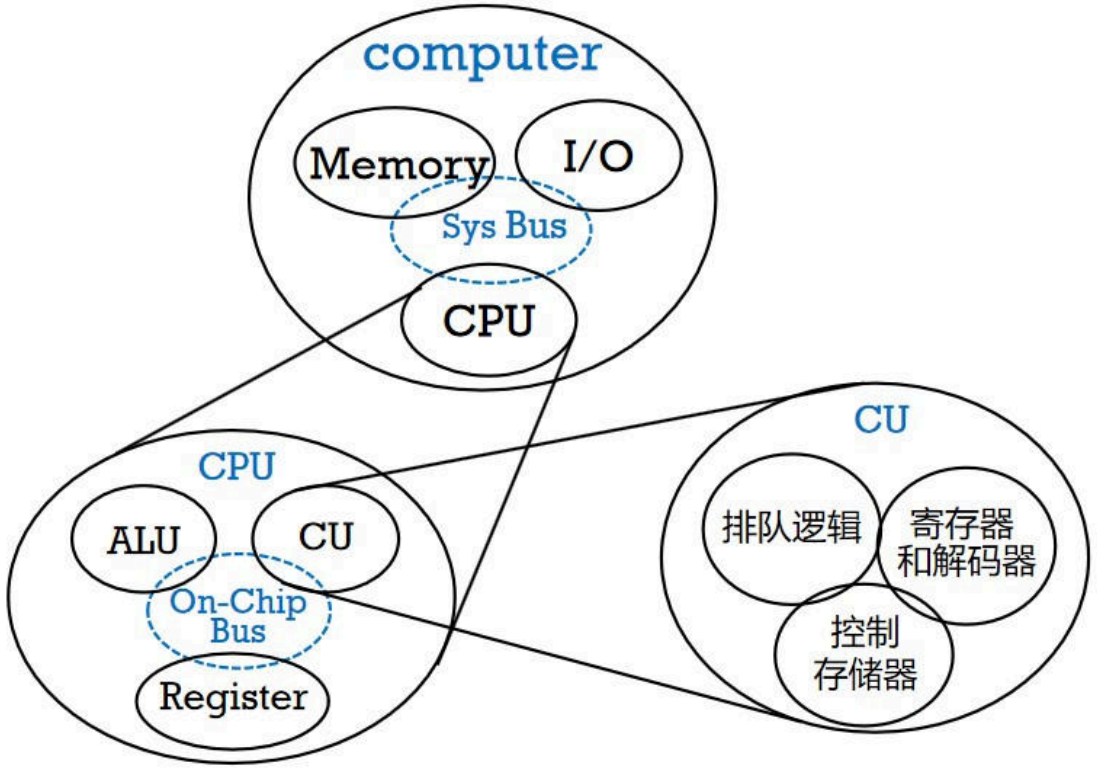
1. 层次化：
   - 将系统分成若干模块，再细分若干模块，直到每个模块都易于理解
2. 模块化：
   - 所有模块都定义好功能和接口，以便于它们之间很容易的相互连接而没有副作用
3. 规整化：
   - 在模块之间寻求一致，以便通用重复使用，减少设计不同模块的数量。

雨课堂 Rain Classroom

三个原则（The Three –Y's）

- 层次化（Hierarchy）



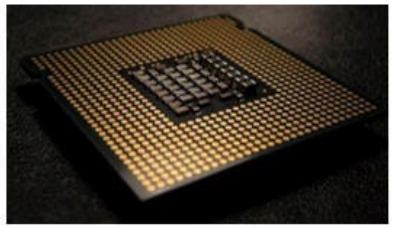


三个原则（The Three –Y's）

- 模块化（Modularity）
  - 每个模块有明确的功能(functions)和接口(interfaces)





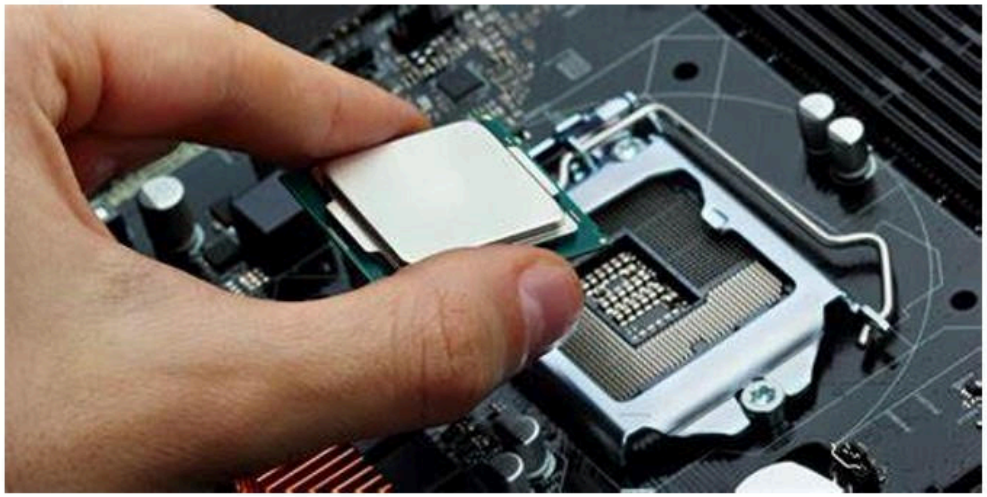






三个原则（The Three –Y's）

- 规整化（Regularity）
  - 易于互换部件

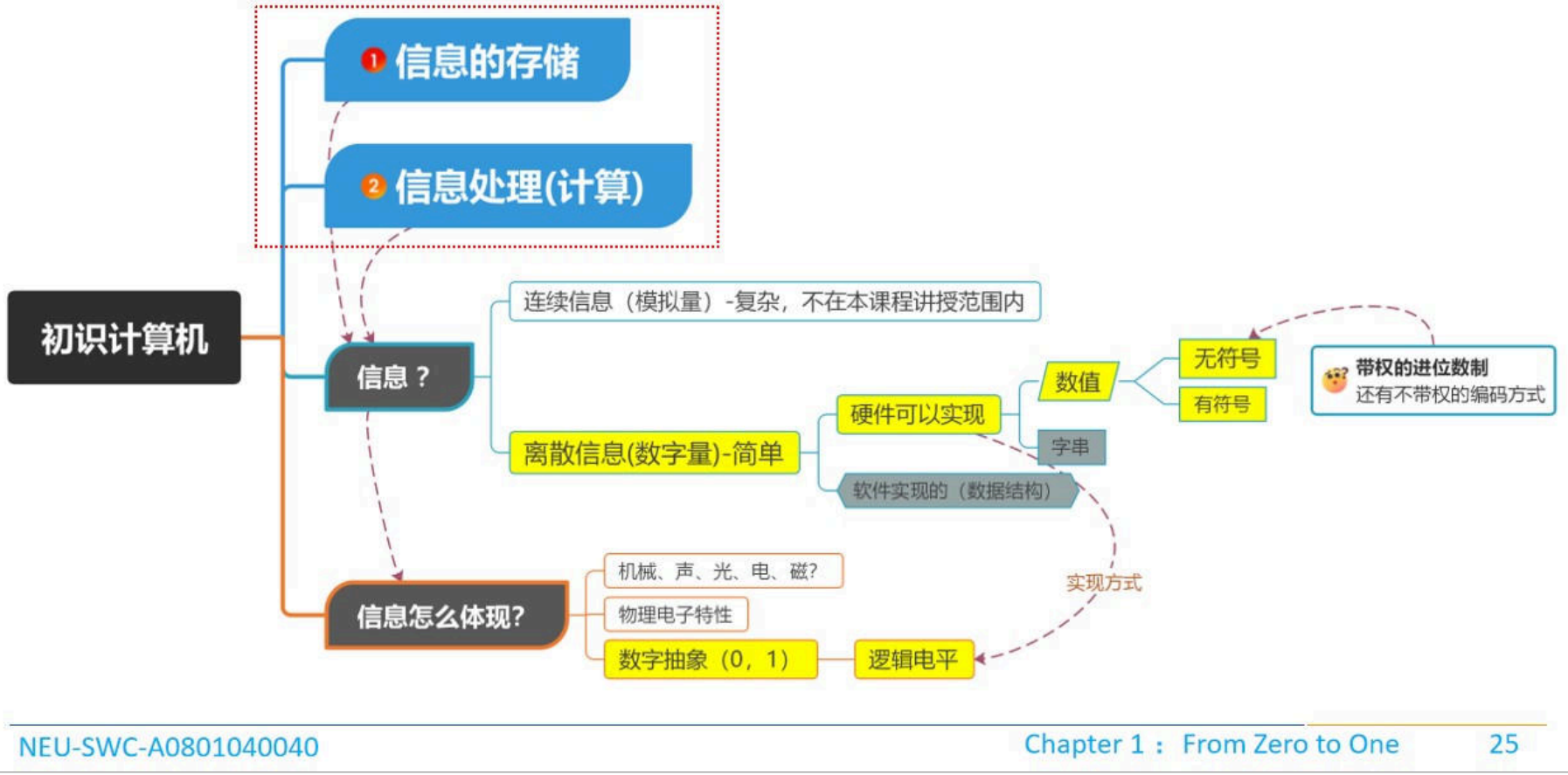


雨课堂 Rain Classroom

Chapter 1 :: Topics

- 驾驭复杂的艺术 (The Art of Managing Complexity)
- **数制系统 (Number Systems)**
- 数字抽象 (The Digital Abstraction)
- 逻辑门 (Logic Gates)
- CMOS晶体管 (CMOS Transistors)
- 功耗 (Power Consumption)

思维导图


1 信息的存储
2 信息处理(计算)
初识计算机
信息？
连续信息（模拟量）-复杂，不在本课程讲授范围内
离散信息(数字量)-简单
硬件可以实现
数值
无符号
有符号
带权的进位数制
还有不带权的编码方式
字串
软件实现的（数据结构）
实现方式
信息怎么体现?
机械、声、光、电、磁?
物理电子特性
数字抽象（0，1）
逻辑电平

思考题（慕课堂讨论）

- 数字系统中常见的编码形式

（1）什么是编码?

（2）为什么要进行编码，编码的目的是什么?

（3）编码的输入是什么？输出是什么？也就是说要对什么东西进行编码？编码之后的形式是什么样的?

（4）你能举例说明都有哪些种类的东西需要进行编码吗?

（5）每一种类输入你能说出一种典型的编码形式吗?



信息表示

- 在计算机系统中，数据的类型有各种各样:
  - integer, float, Boolean, char, string, array, file, graph, table, tree, queue, linked list, stack, vector
- 计算机组成原理课不讨论以上全部信息的表示方法



信息表示

- 确定哪些数据类型用**数据表示**来实现的原则主要有三个：

1. 缩短程序的运行时间；
2. 减少CPU与主存储器之间的通信量；
3. 这种数据表示的通用性和利用率。

- integer, float, Boolean, char, string,  array, file, graph, table, tree, queue, linked list, stack, vector

雨课堂 Rain Classroom

信息表示

- 计算机中主要运行的是“程序”，
  - 程序包括“变量”和“代码”，所以二进制变量和二进制代码是信息表示的基础。

- 最基础的数据表示方式
  - **无符号数（Unsigned numbers）**
  - **有符号数（Signed numbers）**
    - 定点数（Fix-Point Number）
    - 浮点数（Floating-Point Number）
  - **位串（Bit string）**

信息表示

- **无符号数：二进制代码中的很多部分，都是无符号数体现的。如：**
  - 计算机**指令中的操作码**
    - 类似学生的学号，起到检索的作用；
  - 计算机**指令中的地址码**
    - 如，寄存器号/存储器地址；
  - **计数器值/定时器值**（时间），**时钟频率**等
    - 只有正数，没有负数，也可以作一些加减运算等。

雨课堂 Rain Classroom

信息表示

- **符号数**
  - 计算机中的数值和工程计算的数
  - 算数操作指令处理的数值（带符号的整数或浮点数）

- **位串**（比符号数更为广泛）
  - 指令系统中的逻辑操作的操作数：是一种按位操作，操作数无数值意义
  - 标识符（如文本数据等）
  - 控制器中的状态字、标识数和控制码等

多选题 1分

关于“位串（bit string）”的描述，以下哪些是正确的？（多选）

A 其每一位参与逻辑运算（如AND、OR、XOR），但不赋予数值含义

B 可用于表示ASCII文本字符，此时每个字节是8位位串

C 控制器中的状态字、控制码属于位串，每位代表特定控制信号

D 位串可直接参与算术加减运算，并产生带符号结果

雨课堂 Rain Classroom

数制系统——无符号数（Unsigned numbers）

- 带权的位置数制 (Weighted positional number system)
  - 一个数用多位数字表示
  - 每一位赋予一个规定的数值
  - 全部各位数字所赋予的数值之和等于该数的值

- **基数**（Radix）
- **权**（Weight）

带权的位置数制中的“权”

- 某商场要求把1000公斤(kg)货物装成10箱，为了便于顾客在购买1—1000公斤之间任何数目的货物时，都不要打开包装箱便可拿到，问每只箱内各装入多少千克货物。

慕课堂讨论



雨课堂 Rain Classroom

带权的位置数制中的 “权”

9#	8#	7#	6#	5#	4#	3#	2#	1#	0#
489kg	256kg	128kg	64kg	32kg	16kg	8kg	4kg	2kg	1kg
	✔		✔				✔	✔	
	1	0	1	0	0	0	1	1	0

- 想买326公斤货物
- **权** (Weight)

雨课堂 Rain Classroom

Number Systems



Binary numbers

便于机器

1101.101_2

Decimal numbers

便于人

5374.125_{10}

Hexadecimal Numbers

人机兼顾，是二进制的压缩表示法

$2CA.E_{16}$

Number Systems（自学）

- 整数十进制（Integer Decimal numbers）
  - 5374_{10}
  - 基数（Radix）：10，每一位上可能出现的数字符号有10种 (即，0～9)
  - 权（Weight）：10^i，i =0,1,2, ...

- $5374_{10} = 5 \times 10^3 + 3 \times 10^2 + 7 \times 10^1 + 4 \times 10^0$

雨课堂 Rain Classroom

Number Systems（自学）

- 小数十进制（Decimal fraction numbers）
  - 53.74_{10}
  - 基数（Radix）：10，每一位上可能出现的数字符号有10种 (即，0～9)
  - 权（Weight）：10^i，（整数部分：i = 0,1,2, ... ；小数部分：i = -1,-2,-3, ...）

- $53.74_{10} = \underbrace{5 \times 10^1 + 3 \times 10^0}_{\text{整数部分integer part}} + \underbrace{7 \times 10^{-1} + 4 \times 10^{-2}}_{\text{小数部分fractional part}}$

雨课堂 Rain Classroom

Number Systems（自学）

- 整数二进制（Integer Binary numbers）
  - 1101_2
  - 基数（Radix）：2，每一位上可能出现的数字符号有2种 (即，0和1)
  - 权（Weight）：2^i，i =0,1,2, ...

- $1101_2 = 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 13_{10}$

雨课堂 Rain Classroom

Number Systems（自学）

- 小数二进制（Binary fraction numbers）
  - 11.01_2
  - 基数（Radix）：2，每一位上可能出现的数字符号有2种 (即，0和1)
  - 权（Weight）：2^i，（整数部分：i =0,1,2, … 小数部分：i = -1,-2,-3, …）

- $11.01_2 = 1 \times 2^1 + 1 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2} = 3.25_{10}$

  整数部分integer part　小数部分fractional part

Number Systems（自学）

- 整数十六进制（Integer Hexadecimal Numbers）
  - $2CA_{16}$
  - 基数（Radix）:16，每一位上可能出现的数字符号有16种 (即，0～9，A～F)
  - 权（Weight）：16^i，i =0,1,2, ...

- $2CA_{16} = 2 \times 16^2 + 12 \times 16^1 + 10 \times 16^0 = 714_{10}$

雨课堂 Rain Classroom

Number Systems（自学）

- 小数十六进制（Hexadecimal fraction Numbers）
  - $2CA.E_{16}$
  - 基数（Radix）:16，每一位上可能出现的数字符号有16种 (即，0～9，A～F)
  - 权（Weight）：16^i，（整数部分：i =0,1,2, ... 小数部分：i = -1,-2,-3, ...）

- $2CA.E_{16} = \underline{2 \times 16^2 + 12 \times 16^1 + 10 \times 16^0} + \underline{14 \times 16^{-1}} = 714.875_{10}$

  整数部分integer part　　小数部分fractional part

不同数制之间的转换

- Bin/Hex/Oct → Dec

$$N = d_{n-1}d_{n-2}\ldots\ldots d_1 d_0 . d_{-1}d_{-2}\ldots\ldots d_{-m}$$

$$= d_{n-1}\times R^{n-1} + d_{n-2}\times R^{n-2} + \ldots\ldots + d_1\times R^1 + d_0\times R^0$$ (整数部分：每一位的权R^i,i=0,1,2...)

$$+ d_{-1}\times R^{-1} + d_{-2}\times R^{-2} + \ldots\ldots + d_{-m}\times R^{-m}$$ (小数部分：每一位的权R^i,i=-1,-2,-3...)

其中，R 是基数. R = 2/8/16.....

- 非十进制转换至十进制方法：按权展开法，先写成多项式，然后计算十进制结果

雨课堂 Rain Classroom

多选题 1分

关于位置记数法中某一位的“权”，下列描述正确的是：

A 整数部分第 i 位（从右往左，最低位为第0位）的权是 R^{-i}

B 小数部分第 i 位（从左往右，小数点后第1位为第-1位）的权是 R^{i}，其中 $i = -1, -2, -3, \ldots$

C 任意进制数 $d_{n-1}d_{n-2}\ldots d_0.d_{-1}d_{-2}\ldots d_{-m}$ 的十进制值为 $\sum_{i=-m}^{n-1} d_i \times R^i$

D 基数 R 只能取 2、8、10、16 四种值

雨课堂 Rain Classroom

不同数制之间的转换

- Dec → Bin/Hex/Oct
- 分两步：
  - 整数部分：除基取余法
    - 反复除以 2/8/16(基数) 的十进制除法运算
  - 小数部分：乘基取整法
    - 连续乘以2/8/16(基数) 的十进制乘法运算



- Convert 327.8125_{10} and 327.2_{10} to binary

雨课堂 Rain Classroom

整数部分：除基取余

Convert 327_{10} to binary

整数部分	取余数	
2 | 327		
2 | 163	1	……低位
2 | 81	1	
2 | 40	1	
2 | 20	0	
2 | 10	0	
2 | 5	0	
2 | 2	1	
2 | 1	0	
0	1	……高位

第一个得到的就是离小数点最近的位

$(327)_{10}=(101000111)_2$

雨课堂 Rain Classroom

小数部分：乘基取整

- Convert 0.8125_{10} to binary

小数部分		取整数部分	
0.8125	× 2 = 1.625	1	……high
0.625	× 2 = 1.25	1	
0.25	× 2 = 0.5	0	
0.5	× 2 = 1	1	……low

第一个得到的就是离小数点最近的位

$(0.8125)_{10}=(0.1101)_2$

雨课堂 Rain Classroom

小数部分：乘基取整

- Convert 0.2_{10} to binary

小数部分	取整数部分	
$0.2 \times 2 = 0.4$	0	……high
$0.4 \times 2 = 0.8$	0	
$0.8 \times 2 = 1.6$	1	
$0.6 \times 2 = 1.2$	1	
$0.2 \times 2 = 0.4$	0	
$0.4 \times 2 = 0.8$	0	
$0.8 \times 2 = 1.6$	1	
$0.6 \times 2 = 1.2$	1	……low
……	……	



第一个得到的就是
离小数点最近的位

$$(0.2)_{10} = (0.\underline{0011}\ \underline{0011}\ \underline{0011}\ \underline{0011}\ldots\ldots)_2$$

雨课堂 Rain Classroom

不同数制之间的转换

- Bin → Hex/Oct

binary	octal	binary	hex	binary	hex
000	0	0000	0	1000	8
001	1	0001	1	1001	9
010	2	0010	2	1010	A
011	3	0011	3	1011	B
100	4	0100	4	1100	C
101	5	0101	5	1101	D
110	6	0110	6	1110	E
111	7	0111	7	1111	F

雨课堂 Rain Classroom

不同数制之间的转换

- Bin ⟺ Oct
  - $(10110111.01101)_2 = (267.32)_8$

binary: 10, 110, 111 . 011, 01

binary: 010, 110, 111 . 011, 010

octal: 2 6 7 . 3 2

  - $(123.46)_8 = (001, 010, 011.100, 110)_2$

$= (1010011.10011)_2$

雨课堂 Rain Classroom

不同数制之间的转换

- Bin ⟺ Hex
  - $(110110111.01101)_2 = (1B7.68)_{16}$

binary: 1, 1011, 0111 . 0110, 1

binary: 0001, 1011, 0111 . 0110, 1000

Hex: 1 B 7 . 6 8

- $(7AC.DE)_{16} = (0111,1010,1100.1101,1110)_2$

$= (11110101100.1101111)_{16}$

雨课堂 Rain Classroom

带权的位置数制中的 “权”

9#	8#	7#	6#	5#	4#	3#	2#	1#	0#
489kg	256kg	128kg	64kg	32kg	16kg	8kg	4kg	2kg	1kg
✔				✔	✔				✔
1	0	0	0	1	1	0	0	0	1

- 如果想买538公斤货物呢?
- $538_{10} = 10\ 0001\ 1010_2 \neq 10\ 0011\ 0001_2$

雨课堂 Rain Classroom

二进制表达数值个数与取值范围

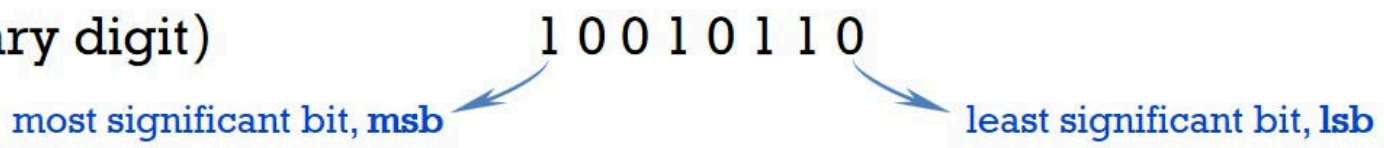
随机点名→ 慕课堂练习

- *N*位十进制数
  - 能表达多少个数?
  - 数域范围是?
  - 例如: 3-位十进制数:
- *N*-位二进制数
  - 能表达多少个数?
  - 数域范围是?
  - 例如: 3-位二进制数:

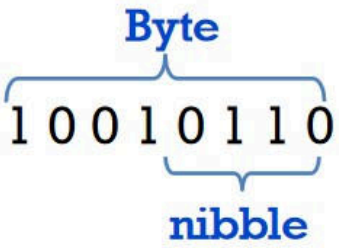
雨课堂 Rain Classroom

几个术语：位Bits，字节Bytes，半字节Nibbles，字word…

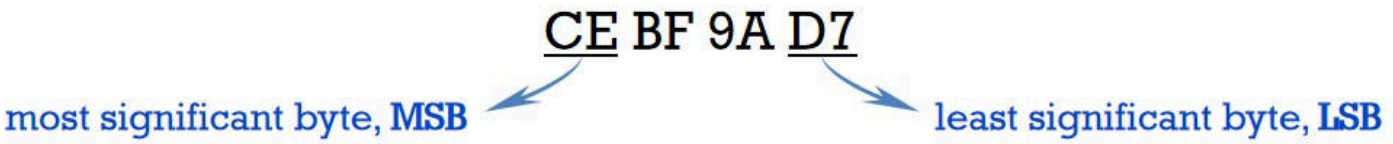
- Bits（binary digit）



- Bytes & Nibbles



- words



微处理器可一次处理的一组数据称为字，字的大小取决于微处理器的结构，比如现在64位是主流，32位也使用了很长时间

2的整数次幂 Powers of Two

$2^0 =$

$2^1 =$

$2^2 =$

$2^3 =$

$2^4 =$

$2^5 =$

$2^6 =$

$2^7 =$

$2^8 =$

$2^9 =$

$2^{10} =$

$2^{11} =$

$2^{12} =$

$2^{13} =$

$2^{14} =$

$2^{15} =$

雨课堂 Rain Classroom

2的整数次幂 Powers of Two

$2^0 = 1$	$2^8 = 256$
$2^1 = 2$	$2^9 = 512$
$2^2 = 4$	$2^{10} = 1024$
$2^3 = 8$	$2^{11} = 2048$
$2^4 = 16$	$2^{12} = 4096$
$2^5 = 32$	$2^{13} = 8192$
$2^6 = 64$	$2^{14} = 16384$
$2^7 = 128$	$2^{15} = 32768$

记住这些2的整数次幂(至少到2^9) → 为了二进制数值的估算

特殊的几个2的整数次幂

- 2^{10} = 1 Kilo ≈ 1000 (1024 ≈ 10^3) 千
- 2^{20} = 1 Mega ≈ 1 million (1,048,576 ≈ 10^6) 百万
- 2^{30} = 1 Giga ≈ 1 billion (1,073,741,824 ≈ 10^9) 十亿
- 2^{40} = 1 Tera ≈ 1 trillion (1,099,511,627,776 ≈ 10^{12}) 万亿

- 记住1K, 1M, 1G的二进制估算值，也是为了结合前面的2的n次幂，来估算二进制数值的大小

多选题 1分

以下关于二进制前缀的近似换算关系，哪些是正确的？（多选）

A $2^{10} \approx 10^3$，对应 Kilo（千）

B $2^{20} \approx 10^6$，对应 Mega（百万）

C $2^{30} \approx 10^9$，对应 Giga（十亿）

D $2^{40} \approx 10^{12}$，对应 Tera（万亿）

估算2的n次幂

- 估算2^{24}的十进制值是多少?

- 32-位二进制可以表达多少个无符号数?

雨课堂 Rain Classroom

估算2的n次幂

- 估算2^{24}的十进制值是多少?
  - $2^4 \times 2^{20} = 16M \approx 16,000,000$（1600万）

- 32-位二进制可以表达多少个无符号数?
  - $2^2 \times 2^{30} = 4G \approx 4,000,000,000$（40亿）

雨课堂 Rain Classroom

填空题 3分

数制转换

- 二进制转换成十进制:
  - $10011_2 \rightarrow$ [填空1] $_{10}$
  - $1101.01_2 \rightarrow$ [填空2] $_{10}$
- 十进制转换成二进制:
  - $47_{10} \rightarrow$ [填空3] $_2$

雨课堂 Rain Classroom

填空题 2分

数制转换

- 十六进制转换成二进制:
  - $4AF_{16} \rightarrow$ [填空1] $_2$
- 十六进制转换成十进制:
  - $4AF_{16} \rightarrow$ [填空2] $_{10}$

第一堂课结束了

有问题同学们可以提问讨论

Chapter 1 :: Topics

- The Art of Managing Complexity (驾驭复杂的艺术)
- Number Systems (数制系统)
- The Digital Abstraction (数字抽象)
- Logic Gates (逻辑门)
- CMOS Transistors (CMOS晶体管)
- Power Consumption (功耗)

信息表示

- **符号数**
  - 计算机中的数值和工程计算的数
  - 算数操作指令处理的数值（带符号的整数或浮点数）

- 有符号数
  - 定点数 Fix-Point Number
  - 浮点数 Floating-Point Number（第5章介绍）

定点、浮点的“**点**”都指二进制数中的**小数点**。

注意：小数点均为隐含表示，不占位。

信息表示

- 定点数
  - 大多数计算机中的定点数表示，都表现为定点小数(**纯小数**)和定点整数(**纯整数**)
  - 这样做的好处是当**数据表示范围**和**精度**可以通过纯小数和纯整数来满足时，计算效率更高。.



有符号二进制数

- 如何表示数字的**符号**?
  - '0' 表示正号
  - '1' 表示负号
  - 单独设置 1 位符号位，置于最高有效位位置（**msb**，the **m**ost **s**ignificant **b**it）
- 数值如何表示呢?
  - 原码 Sign-Magnitude Numbers（sign-mag）
  - 补码 Two´s Complement Numbers（2's compl）
  - 反码 One´s Complement Numbers（1's compl）

雨课堂 Rain Classroom

原码表示 Sign-Magnitude Numbers

- 1-bit符号位, N-1位数值位
- 符号位在最左侧最高位上（**msb**，left-most）
  - 正数: 符号位 = 0
  - 负数: 符号位 = 1

- *M*位原码的数学表达式： $A:\{a_{n-1},a_{n-2},\cdots a_2,a_1,a_0\}$

$$A=(-1)^{a_{n-1}}\sum_{i=0}^{n-2}a_i 2^i$$

雨课堂 Rain Classroom

原码表示 Sign-Magnitude Numbers

- 例如, 用 4-bit 原码表达 ± 6:
  - +6 = 0,110
  - - 6 = 1,110

手写整数原码时，用逗号“,”作为分隔符，区分符号位和数值位

- 例如，用 4-bit 原码表达 ± 0.75:
  - +0.75 = 0.110
  - - 0.75 = 1.110

手写小数原码时，用句点“.”作为分隔符，区分符号位和数值位，不代表小数点

- 在计算机中：存储 +6 和 +0.75，都是0110；存储 -6 和 -0.75，都是1110

雨课堂 Rain Classroom

填空题 2分

- 纯小数原码表示练习

  - $X_1 = +0.1011011$ （带正负号的原值，叫“**真值**”）
  - $X_{1\ sign\ mag} =$ [填空1] （在一定位数空间存储的、符号数值化的，叫“**机器值**”）

  - $X_2 = -0.1011011$ （带正负号的原值，叫“真值”）
  - $X_{2\ sign\ mag} =$ [填空2] （机器值，**machine value**）

填空题 2分

- 纯整数原码表示练习

  - $X_1 = +1011011$ （带正负号的原值，叫“**真值**”）
  - $X_{1\ sign\ mag}$ = [填空1] （在一定位数空间存储的、符号数值化的，叫“**机器值**”）

  - $X_2 = -1011011$ （带正负号的原值，叫“真值”）
  - $X_{2\ sign\ mag}$ = [填空2] （机器值，**machine value**）

原码的表示范围

- 特殊示例，假设计算机使用4位原码来存储有符号数: +6
  - $[+6]_{\text{sign mag}} = 0,110$
  - 那么两个+6相加：(+6) + (+6)

```
   0,110                ,110
 + 0,110              + ,110
 = 1,100 ➡ -4         = 0,100 ➡ +4
```

- N-bit 原码的表示范围是多少？
  - 例如，4-bit 整数原码表示范围是[-7, +7], 可以表示15个数

0,000	+0
0,001	+1
0,010	+2
0,011	+3
0,100	+4
0,101	+5
0,110	+6
0,111	+7
1,000	-0
1,001	-1
1,010	-2
1,011	-3
1,100	-4
1,101	-5
1,110	-6
1,111	-7

$2^{n-1}-1$ 最大值

$-(2^{n-1}-1)$ 最小值

原码的表示范围

- N-bit 纯整数原码
  - 表示范围：$-(2^{n-1}-1) \sim +(2^{n-1}-1)$
  - 可表达的数的个数：$2^n - 1$
- N-bit 纯小数原码
  - 表示范围：$-(2^{n-1}-1)/2^{n-1} \sim +(2^{n-1}-1)/2^{n-1}$

    即，$-(1-1/2^{n-1}) \sim +(1-1/2^{n-1})$
  - 可表达的数的个数：$2^n - 1$

0.000	+0		
0.001	+1/8		
0.010	+2/8		
0.011	+3/8		
0.100	+4/8		
0.101	+5/8		
0.110	+6/8		
0.111	+7/8	$1-1/2^{n-1}$	Max
1.000	-0		
1.001	-1/8		
1.010	-2/8		
1.011	-3/8		
1.100	-4/8		
1.101	-5/8		
1.110	-6/8		
1.111	-7/8	$-(1-1/2^{n-1})$	Min

单选题 1分

若某计算机使用8位原码表示有符号整数，则其能表示的最小整数是?

A -127

B -128

C -255

D 0

填空题 3分

假设某机器字长是 8

- 纯整数数据用原码表示：
- 一个字长能表达的数据范围是：[填空1]—— [填空2]

  一个字长能表达的数的数量是：[填空3]
- 纯小数数据用原码表示：
- 一个字长能表达的数据范围是：[填空4] —— [填空5]

  一个字长能表达的数的数量是：[填空6]

雨课堂 Rain Classroom

补码的提出（2's complement）

- 问题示例:
  - 使用原码表示的二进制数据进行加减法操作, 例如： -6 + 6:

    1,110
    + 0,110
    10,100 (错误：符号位进位的“1”被忽略掉，结果是0,100，是+4)

- **用原码做加减法有问题:**
  - 两个正数相加，如果没有溢出情况，还可以；若一正一负，肯定有问题；若两个都是负数也有问题。
- **问题原因:**
  - 符号位没有权值，不能参与运算；± 0有两种表达: 1000, 0000

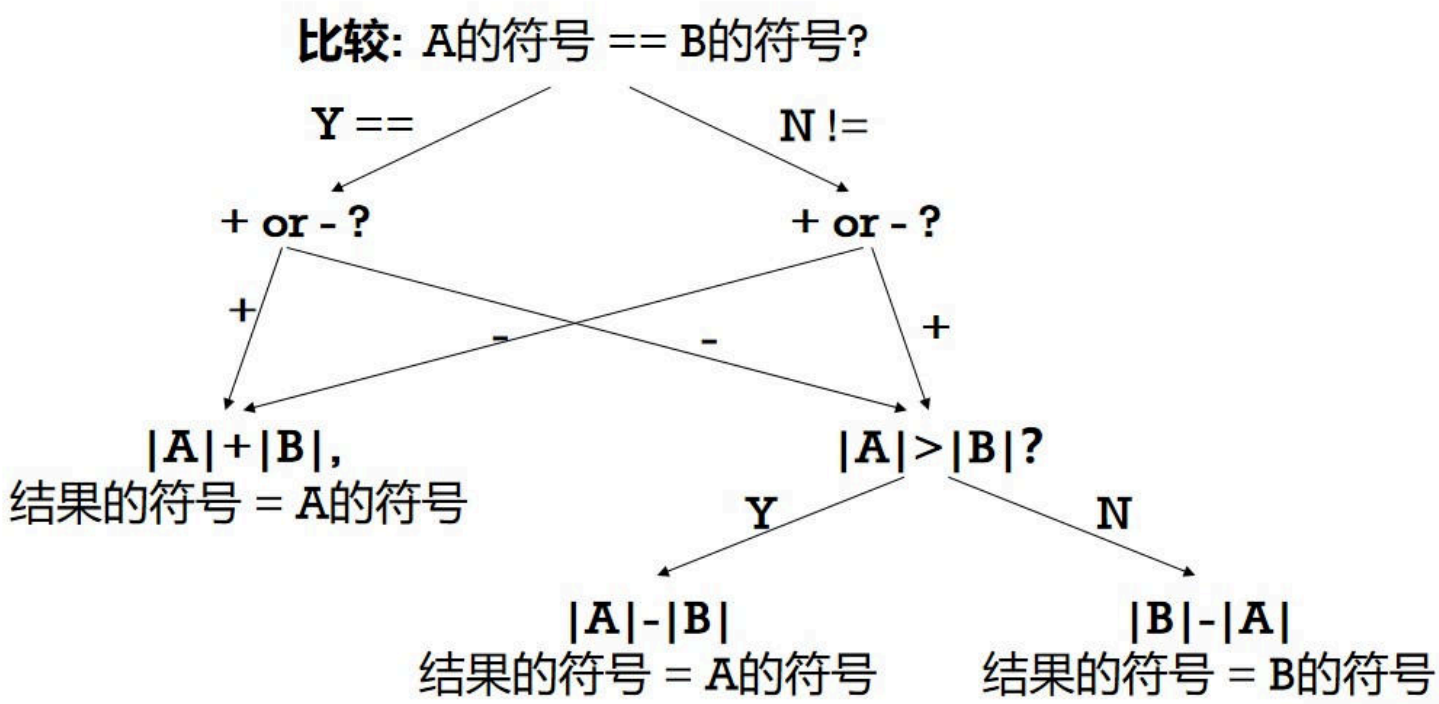
补码的提出（2's complement）

- **原码特点**
  - 优点： 
    - 表达简单，易于进行乘除法(multiply/divide)运算
  - 缺点： 
    - 存在+0，-0 两种表达。其实0是一个数，有两种表达会造成麻烦；
    - 对于加减(**add/sub**)运算非常苦难
- 所以，才提出补码表示（**2's complement** representation）

雨课堂 Rain Classroom

补码的提出（2's complement）

- 例如，$A_{sign\text{-}mag}$ +/- $B_{sign\text{-}mag}$



补充知识点

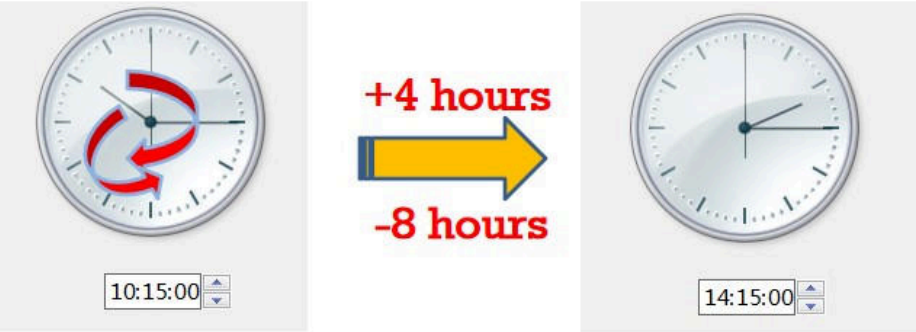
计算机做减法
为什么不需要减法器？



用加法代替减法

- 简化加减法操作，把减法变加法，那么所有的加减法操作就变成了加法

- 生活中我们有实例



用加法代替减法

$$\begin{array}{r} 113 \\ -\ 76 \\ \hline ??? \end{array}$$



雨课堂 Rain Classroom

用加法代替减法

$$
\begin{array}{rr}
 & 113 \\
- & 76 \\
\hline
 & ??? \\
\end{array}
$$



$$
\begin{array}{rr}
 & 113 \\
+ & 999 + 1 \\
- & 76 \\
- & 1000 \\
\hline
 & ??? \\
\end{array}
$$

雨课堂 Rain Classroom

用加法代替减法

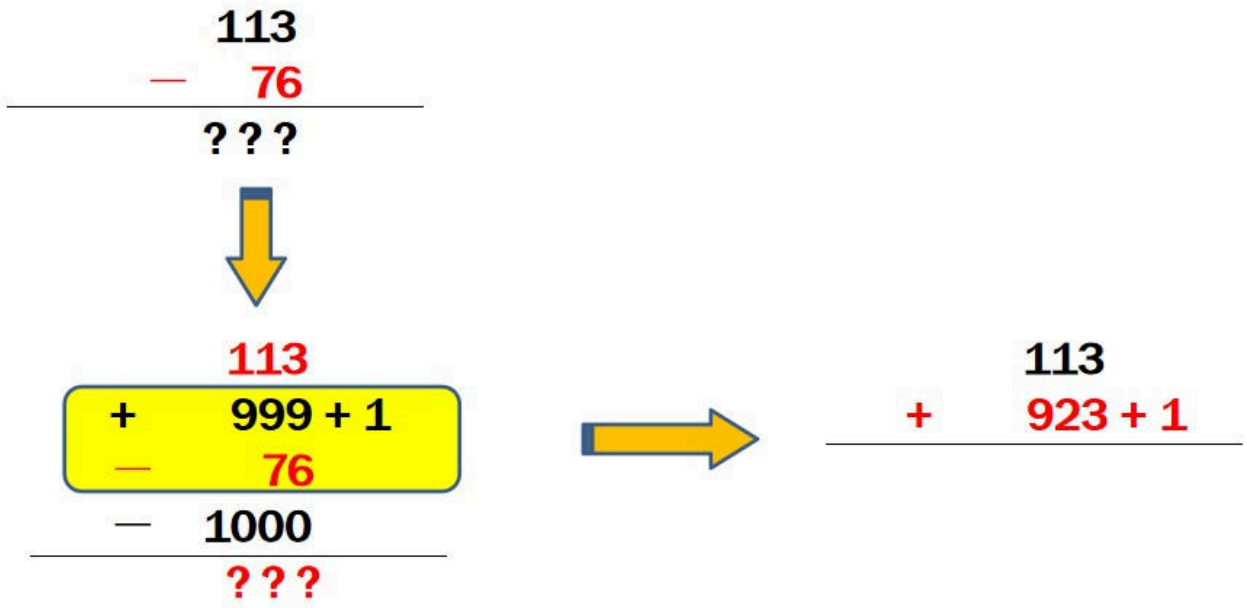
$$
\begin{array}{r}
113 \\
-\ 76 \\
\hline
??? \\
\end{array}
$$



$$
\begin{array}{r}
113 \\
+\ 999+1 \\
-\ 76 \\
\hline
-\ 1000 \\
\hline
??? \\
\end{array}
$$

雨课堂 Rain Classroom

用加法代替减法



用加法代替减法

$$
\begin{array}{r}
113 \\
-\ 76 \\
\hline
??? \\
\end{array}
$$



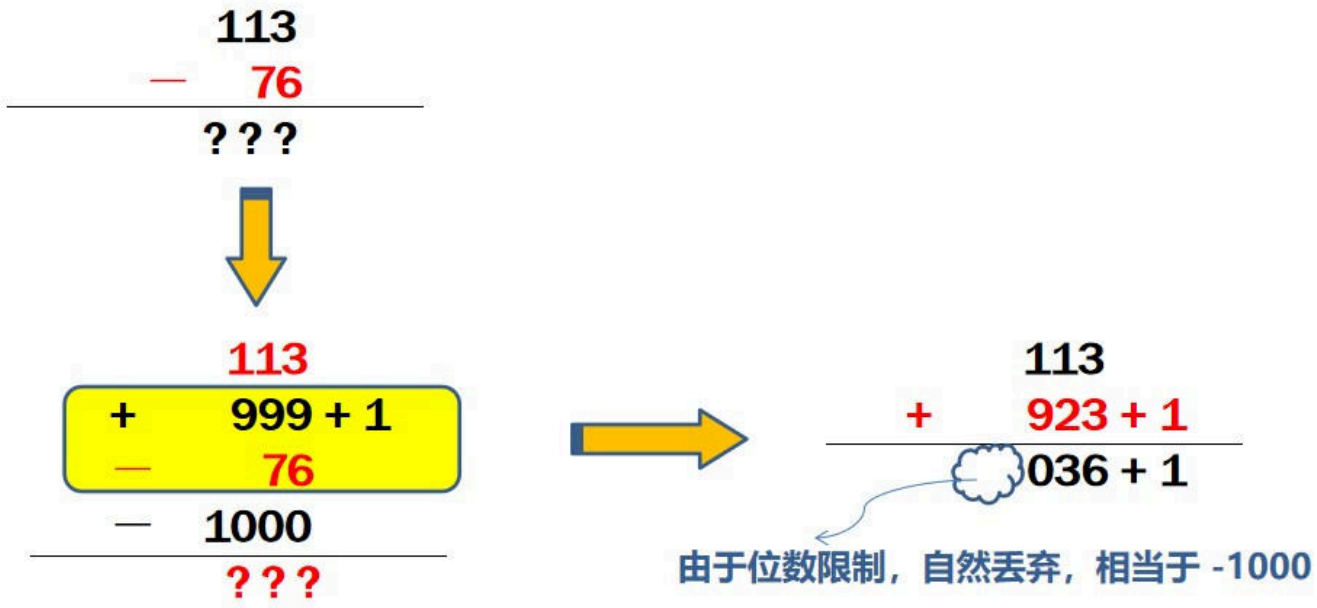
$$
\begin{array}{r}
113 \\
+\ 999+1 \\
-\ 76 \\
\hline
-\ 1000 \\
\hline
??? \\
\end{array}
$$



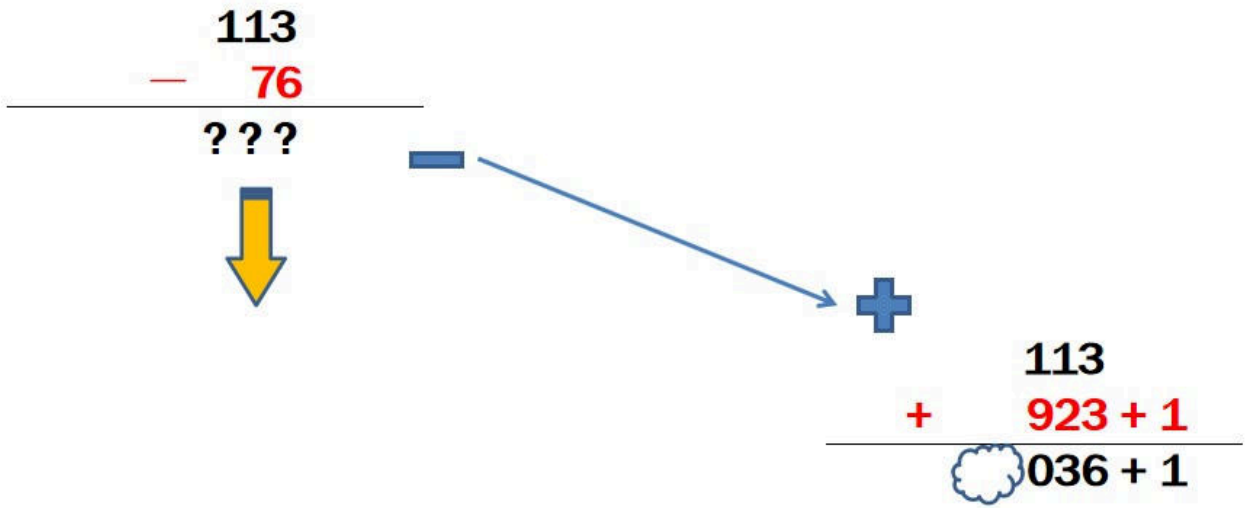
$$
\begin{array}{r}
113 \\
+\ 923+1 \\
\hline
1\,036+1 \\
\end{array}
$$

雨课堂 Rain Classroom

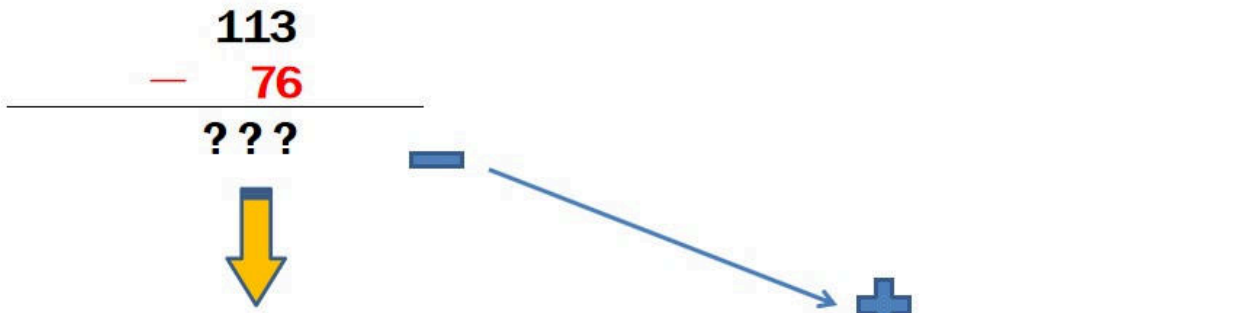
用加法代替减法



用加法代替减法



用加法代替减法



这里就称“924”是“-76”
相对于1000的补码表示。

可以看出负数的补码就是负数本身加上“模”。

单选题 1分

负数的补码在数学上等于什么?

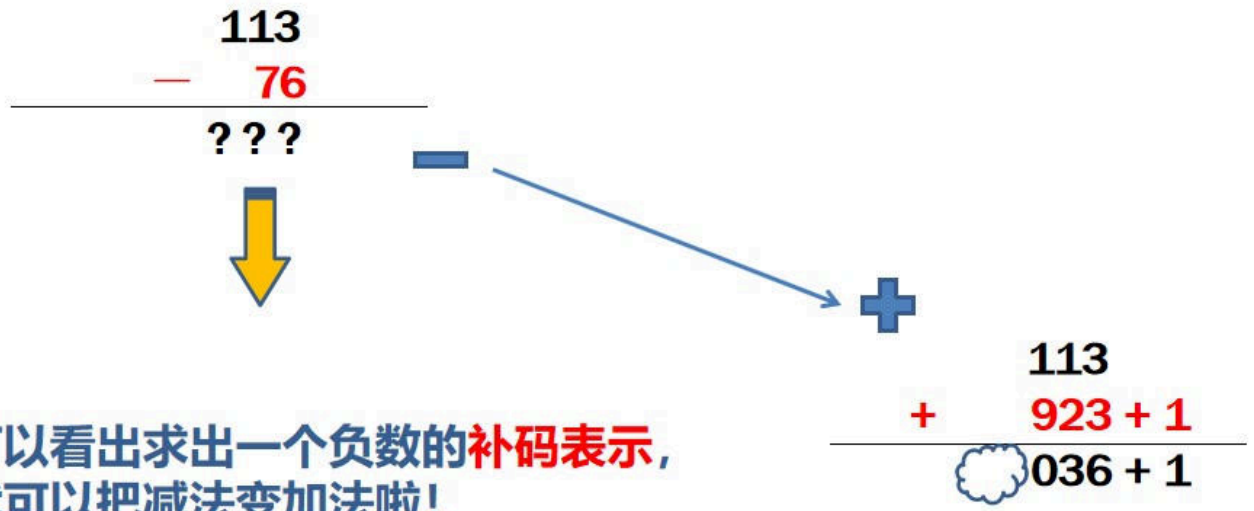
A 负数的绝对值

B 负数本身加上模（如1000或2^N）

C 负数的原码按位取反

D 负数的反码加1

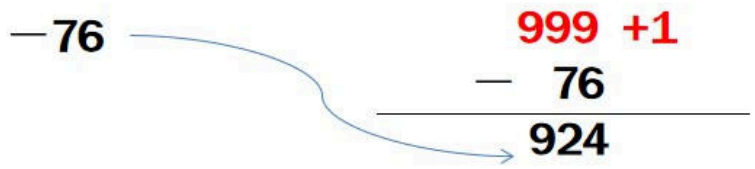
用加法代替减法



可以看出求出一个负数的**补码表示**，
就可以把减法变加法啦！

如何找到二进制的补码表示?

- 找到一个十进制负数的补码，得用负数本身加上“模”



- 对于计算机二进制负数的补码，也是这么麻烦吗?



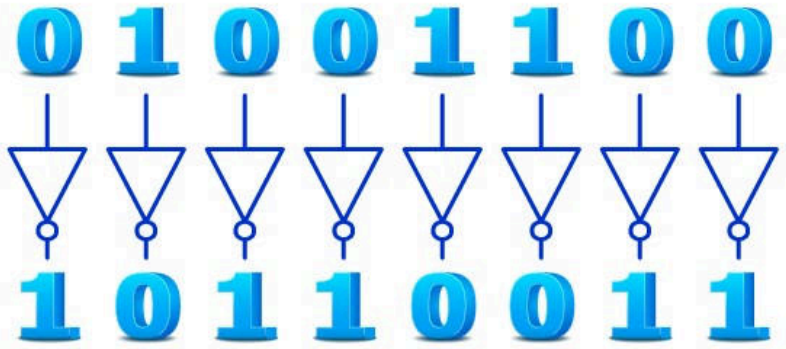
不，不是的

如何找到二进制的补码表示?

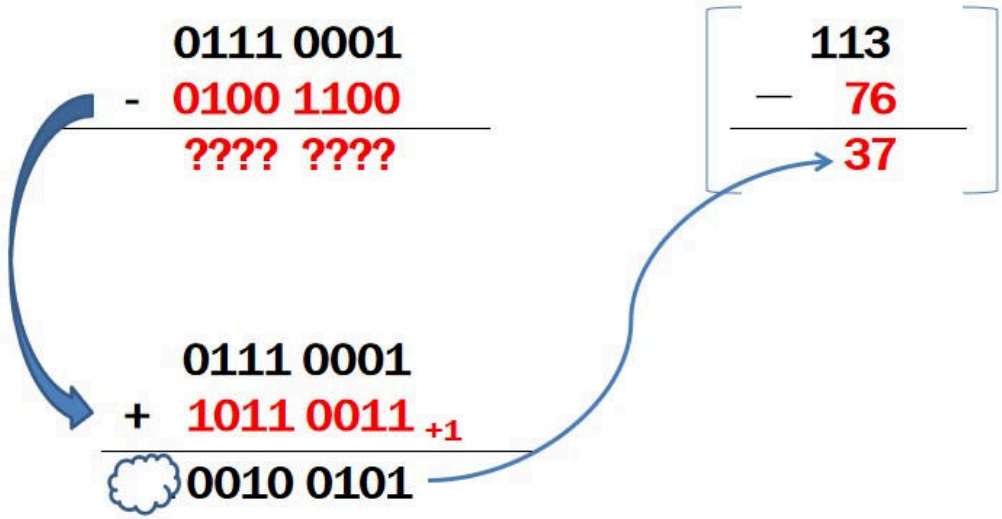
- 对于计算机二进制来说，找到补码非常简单!

$-76 \longrightarrow 76_{10} = 0100\ 1100_2$

$$
\begin{array}{r}
1111\ 1111 \\
-\ 0100\ 1100 \\
\hline
1011\ 0011
\end{array}
\quad {}^{+1} \quad {}^{+1}
$$



用加法代替减法



补码表示消灭负数

- 就象我们前面所演示给大家看的，补码表示的发现其实是为了消灭减法。
- 正数根本不需要什么补码。

所以：

- 正数补码：和原码一样，符号用 0 表示，数值用二进制表示；
- 负数补码：符号用 1 表示，**数值部分**按位取反，**末位**加1.

无论纯整数/纯小数，甚至是既有整数又有小数的定点数

雨课堂 Rain Classroom

补码表示示例

- 例如, 用 4-bit 补码表达 ± 6:
  - +6 = 0,110
  - - 6 = 1,010

手写整数原码时，用逗号“,”作为分隔符，区分符号位和数值位

- Example, 4-bit sign-mag representations of ± 0.75:
  - +0.75 = 0.110
  - - 0.75 = 1.010

手写小数原码时，用句点“.”作为分隔符，区分符号位和数值位，不代表小数点

- 在计算机中：存储 +6 和 +0.75，都是0110；存储 -6 和 -0.75，都是1010

雨课堂 Rain Classroom

填空题 2分

纯小数补码表示练习

- $X_1 = +0.1011011$ （带正负号的原值，叫“**真值**”）
- $X_{1\ 2's\ compl} =$ [填空1] （在一定位数空间存储的、符号数值化的，叫“**机器值**”）

- $X_2 = -0.1011011$ （带正负号的原值，叫“真值”）
- $X_{2\ 2's\ compl} =$ [填空2] （机器值，**machine value**）

填空题 2分

纯整数补码表示练习

- $X_1 = +\ 1011011$ （带正负号的原值，叫“**真值**”）
- $X_{1\ 2's\ compl} =$ [填空1] （在一定位数空间存储的、符号数值化的，叫“**机器值**”）

- $X_2 = -\ 1011011$ （带正负号的原值，叫“真值”）
- $X_{2\ 2's\ compl} =$ [填空2] （机器值，**machine value**）

雨课堂 Rain Classroom

补码表示

- 1-bit符号, N-1位数值位
- 最左侧的最高位(The most significant bit)标识符号
  - 1 表示 负(negative), 0 表示 正(positive)
  - **最高位(msb)的符号位有权值：** -2^{N-1}
- N位补码的数学表达式： $A:\{a_{n-1},a_{n-2},\cdots a_2,a_1,a_0\}$

$$A=a_{n-1}\left(-2^{n-1}\right)+\sum_{i=0}^{n-2}a_i 2^i$$

雨课堂 Rain Classroom

补码的表示范围

- 特殊示例，假设计算机使用4位补码来存储有符号数: +6
  - $[+6]_{2's\ compl} = 0,110$
  - 那么两个+6相加：(+6) + (+6)

$$\begin{array}{r} 0,110 \\ +\ 0,110 \\ \hline =\ 1,100 \end{array} \Rightarrow -4$$

0,000	0
0,001	+1
0,010	+2
0,011	+3
0,100	+4
0,101	+5
0,110	+6
0,111	+7
1,000	-8
1,001	-7
1,010	-6
1,011	-5
1,100	-4
1,101	-3
1,110	-2
1,111	-1

$2^{n-1}-1$最大值

奇异值，无相反数 → -2^{n-1} 最小值

- N-bit 补码的表示范围是多少？
  - 例如，4-bit 整数补码表示范围是[-8, +7], 可以表示16个数

雨课堂 Rain Classroom

补码表示

- 在一定字长空间内存储的补码，其表达的负数比正数多一个
  - **因为负数没有 -0.**

- 最小的负数是 $10...000_2 = -2^{N-1}$ ，它又叫**怪异数**(weird number).
  - 这个负数，按位取反 (即 $01...111_2$)，末位加1，结果还是自己($10...000_2$)
  - 所以，它没有相对应的正数。

补码的表示范围

- N-bit 纯整数补码
  - 表示范围：$-2^{n-1} \sim +(2^{n-1}-1)$
  - 可表达的数的个数：2^n

- N-bit 纯小数补码
  - 表示范围：$-2^{n-1}/2^{n-1} \sim +(2^{n-1}-1)/2^{n-1}$

    即 $-1 \sim +(1-1/2^{n-1})$
  - 可表达的数的个数：2^n

0.000	+0		
0.001	+1/8		
0.010	+2/8		
0.011	+3/8		
0.100	+4/8		
0.101	+5/8		
0.110	+6/8		
0.111	+7/8	$1-1/2^{n-1}$	Max
1.000	-1	-1	Min
1.001	-1/8		
1.010	-2/8		
1.011	-3/8		
1.100	-4/8		
1.101	-5/8		
1.110	-6/8		
1.111	-7/8		

填空题 3分

假设某机器字长是 8

- 纯整数数据用补码表示：

  一个字长能表达的数据范围是：[填空1]—— [填空2]

  一个字长能表达的数的数量是：[填空3]

- 纯小数数据用补码表示：

  一个字长能表达的数据范围是： [填空4] —— [填空5]

  一个字长能表达的数的数量是： [填空6]

雨课堂 Rain Classroom

补码加法

- 既然补码的发现或者发明，是为了计算加减法，那我们就来看看计算是否正确，会不会错呢？永远都对吗？
- 核心规则
  - 把所有操作数转换成相同位数的二进制补码；
  - 按普通二进制加法逐位相加；
  - 丢弃最高位产生的进位；
  - 结果仍为二进制补码，按需转回十进制。

雨课堂 Rain Classroom

多选题 1分

使用补码进行加法运算时，以下哪些步骤是正确的？（多选）

A 将所有操作数转换为相同位数的二进制补码

B 按普通二进制加法规则逐位相加

C 保留最高位产生的进位，并将其作为符号位扩展

D 丢弃最高位产生的进位，结果仍为补码表示

补码加法

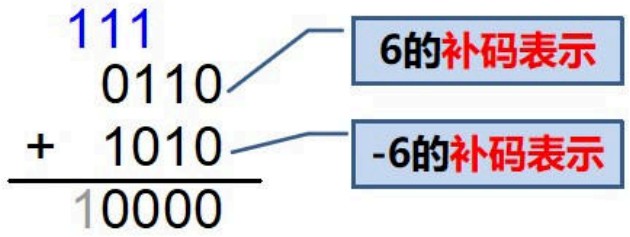
- 使用补码表示，计算加法： (+6) + (-6)

- 使用补码表示，计算加法： (-2) + (+3)

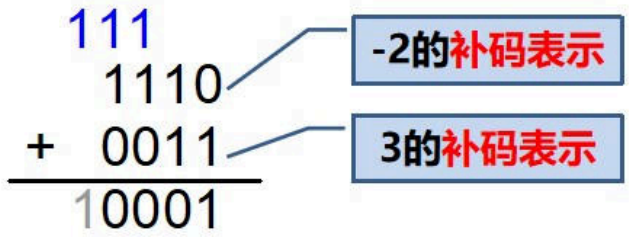
雨课堂 Rain Classroom

补码加法

- 使用补码表示，计算加法： (+6) + (-6)



- 使用补码表示，计算加法： (-2) + (+3)



补码减法

- 使用补码表示，计算减法：(+6) - (+4)

- 使用补码表示，计算减法：(-2) - (-3)

补码减法

- 使用补码表示，计算减法：(+6) - (+4)



- 使用补码表示，计算减法：(-2) - (-3)



求补 Taking the Two's Complement

- "求补"概念：求一个数的相反数；
  - 前提条件：这个数是补码表示；
  - 求补结果：即相反数（也是补码表示）
- "求补"方法：
  - 无论真值是正是负，都把其补码表示的数，按位取反，末位加1
- 例如：$[4]_{2'S\ COMPL} = 0100$，　　$[-3]_{2'S\ COMPL} = 1101$

1,011	按位取反	
+ 1	末位加1	
1,100 = -4_{10}		

0,010	按位取反	
+ 1	末位加1	
0,011 = 3_{10}		

单选题 1分

如果01011是5位补码表示的数字11，那么-11的5位补码表示是：

A 11011

B 10011

C 10101

D 01011

E 10100

单选题 1分

如果01001是5位无符号数9的机器表示，那么9的5位补码表示是

A 10110

B 10111

C 10101

D 01001

E 01010

求补 Taking the Two's Complement

- 对一个特殊的数**求补**，例如: $[0]_{2'S\ Compl} = 0000$

$$\begin{array}{rl} & 1,111 \quad \text{按位取反} \\ + & \quad 1 \quad \text{末位加1} \\ \hline & 0,000 = 0_{10} \end{array}$$

- 发现 +0 的相反数，-0的补码仍然是“全0”(000...000)，这与原码表示不同。
- 在补码系统中，没有单独的-0。 零被认为是正数，因为其符号位为0

雨课堂 Rain Classroom

反码 One's complement representation

- **是过渡编码**
  - **原码变补码的中间结果**
  - 由于正数原码与补码相同，所以**正数反码也与原码相同**
  - **负数反码：符号为1，原码数值按位取反**
- 例如，**小数反码**表示：
  - $X_1 = +0.1011011$, $[X_1]_{1's\ compl} = 0.1011011$
  - $X_2 = -0.1011011$, $[X_2]_{1's\ compl} = 1.0100100$
- 例如，**整数反码**表示：
  - $X_3 = +1011011$, $[X_3]_{1's\ compl} = 0,1011011$
  - $X_4 = -1011011$, $[X_4]_{1's\ compl} = 1,0100100$
- $[+0]_{1's\ compl} = 00000000$; $[-0]_{1's\ compl} = 11111111$

雨课堂 Rain Classroom

课后练习题

假设机器字长是8-bits，写出下列各数的原码sign-mag, 反码1's compl, 补码2's compl 表示, 其中msb是最高位(又称符号位), lsb是最低位.

如果是小数，小数点在msb之后；如果是整数，小数点在lsb之后。

(1) –35/64

(2) 23/128

(3) –127

(4) – 1 (纯小数存储数据)

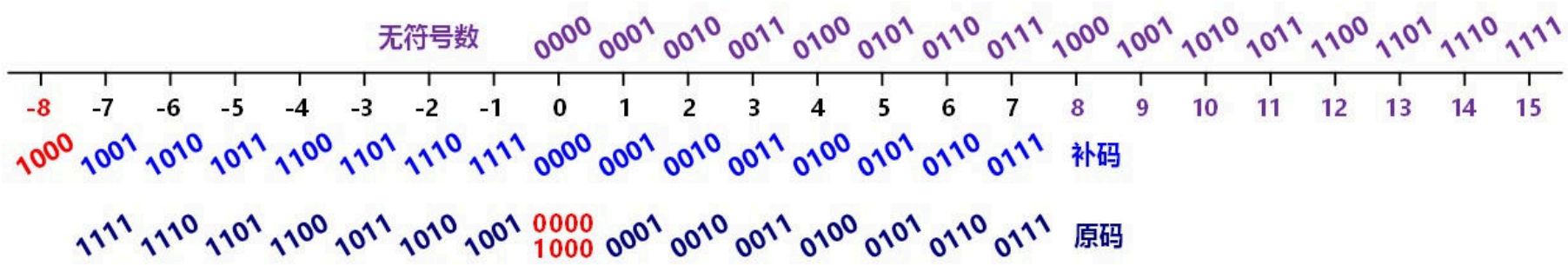
(5) – 1 (纯整数存储数据)

雨课堂 Rain Classroom

定点数小结

数制系统	表示范围（纯整数）	表示范围（纯小数）
无符号数 Unsigned	$[0, 2^N-1]$	--------
原码表示 Sign-Magnitude	$[-(2^{N-1}-1), 2^{N-1}-1]$	$[-(1-1/2^{N-1}), 1-1/2^{N-1}]$
反码表示 One's Complement	$[-(2^{N-1}-1), 2^{N-1}-1]$	$[-(1-1/2^{N-1}), 1-1/2^{N-1}]$
补码表示 Two's Complement	$[-2^{N-1}, 2^{N-1}-1]$	$[-1, 1-1/2^{N-1}]$

例如，4-bit 数据表示：



位宽扩展 Increasing Bit Width

- 在计算机中，数据存储存在一个前提条件：
  - 数字系统只能**基于固定位数**的二进制位开展运算。
  - 例如：32 位 或者 64 位

- 将数值从 N 位扩展至 M 位 (M > N)，有两种扩展方式：
  - 符号扩展（Sign-extension）
  - 零扩展（Zero-extension）

零扩展 Zero-Extension

- 其实我们一直在无意识地使用零扩展：
  - 无符号短整型需要扩展为更长位宽，以适配目标操作数的位长要求时；
  - 执行移位等位运算，且不希望改变数值正负属性时。
- **实现方法**：向“最高有效位（msb）”填充 0

- 例如：
  - 原始 8-bit 数据：0010 1111
  - 零扩展至16-bit 后：0000 0000 0010 1111

雨课堂 Rain Classroom

符号扩展 Sign-Extension

- 符号扩展用于对**有符号整数**进行**位宽扩展**。
- 实现方法：将符号位复制填充到新增的最高有效位，数值大小将保持不变。
- **示例 1:**
  - 十进制 - 5 的 4 位补码表示：　1011
  - 经符号扩展8位后的结果: 11111011
- **示例 2:**
  - 原始 8-bit 数据：　　-81_{10} = 1010 1111
  - 符号扩展至16-bit 后：1111 1111 1010 1111

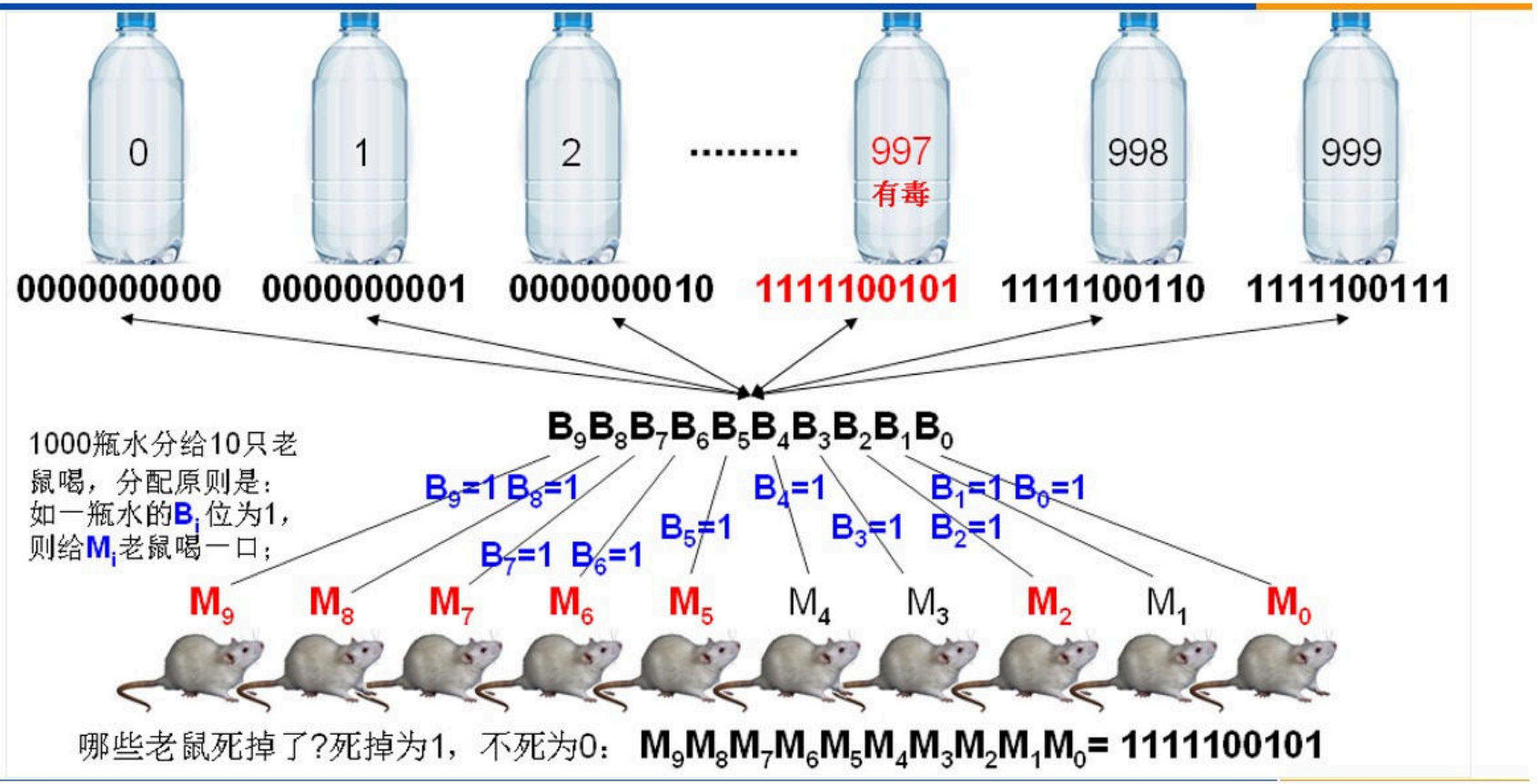
雨课堂 Rain Classroom

二进制数字系统应用

- 有1000瓶水，其中有一瓶有毒。小白鼠只要尝一点点带毒的水，24小时后就会死亡，问：至少需要多少只小白鼠，才能在24小时内鉴别出哪瓶水有毒？



二进制数字系统应用


0
1
2
.....
997
有毒
998
999
0000000000
0000000001
0000000010
1111100101
1111100110
1111100111
$B_9B_8B_7B_6B_5B_4B_3B_2B_1B_0$
1000瓶水分给10只老鼠喝，分配原则是：
如一瓶水的B_i位为1，
则给M_i老鼠喝一口；
B_9=1 B_8=1
B_7=1 B_6=1
B_5=1
B_4=1
B_3=1 B_2=1
B_1=1 B_0=1
M_9 M_8 M_7 M_6 M_5 M_4 M_3 M_2 M_1 M_0
哪些老鼠死掉了?死掉为1，不死为0：
$M_9M_8M_7M_6M_5M_4M_3M_2M_1M_0$= 1111100101

第二堂课结束了

有问题同学们可以提问讨论

雨课堂 Rain Classroom

Chapter 1 :: Topics

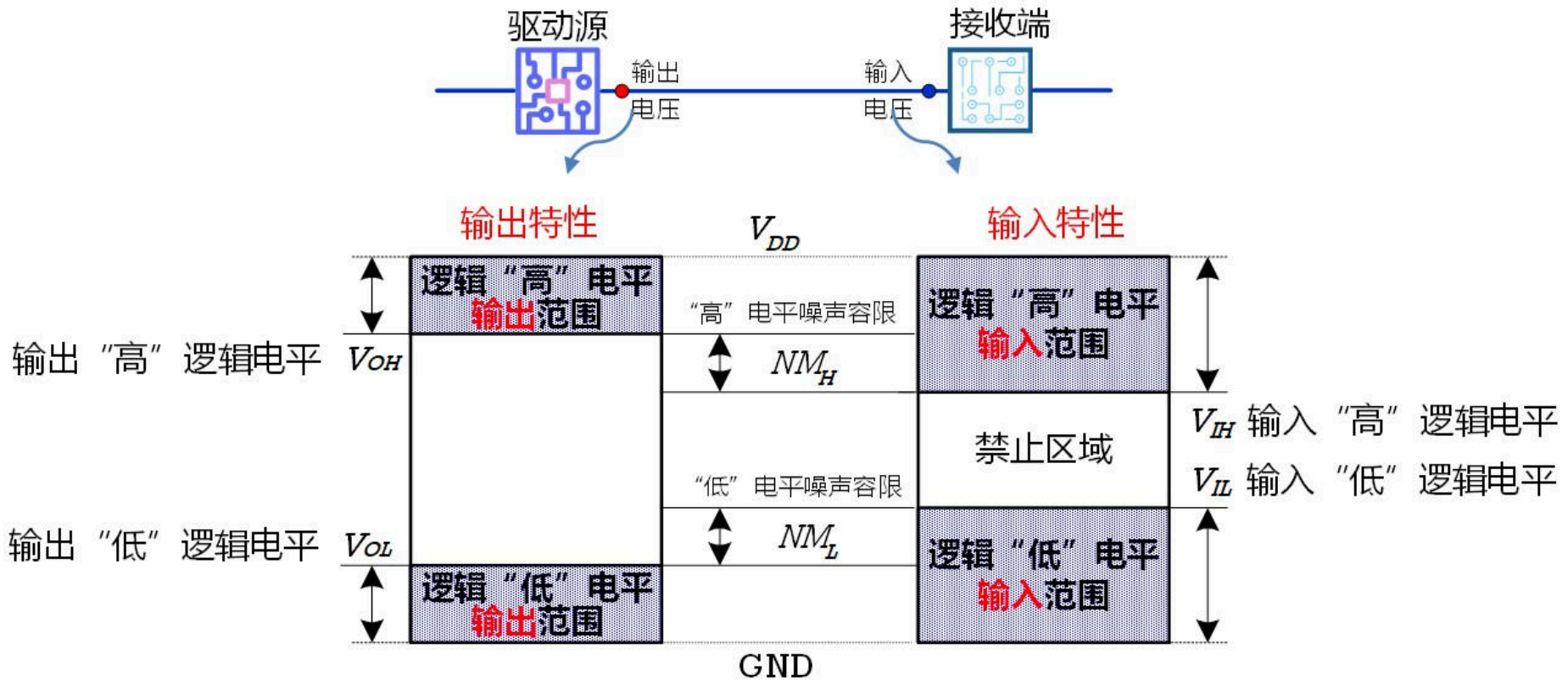
- 驾驭复杂的艺术 (The Art of Managing Complexity)
- 数制系统 (Number Systems)
- **数字抽象 (The Digital Abstraction)**
- 逻辑门 (Logic Gates)
- CMOS晶体管 (CMOS Transistors)
- 功耗 (Power Consumption)

数字抽象

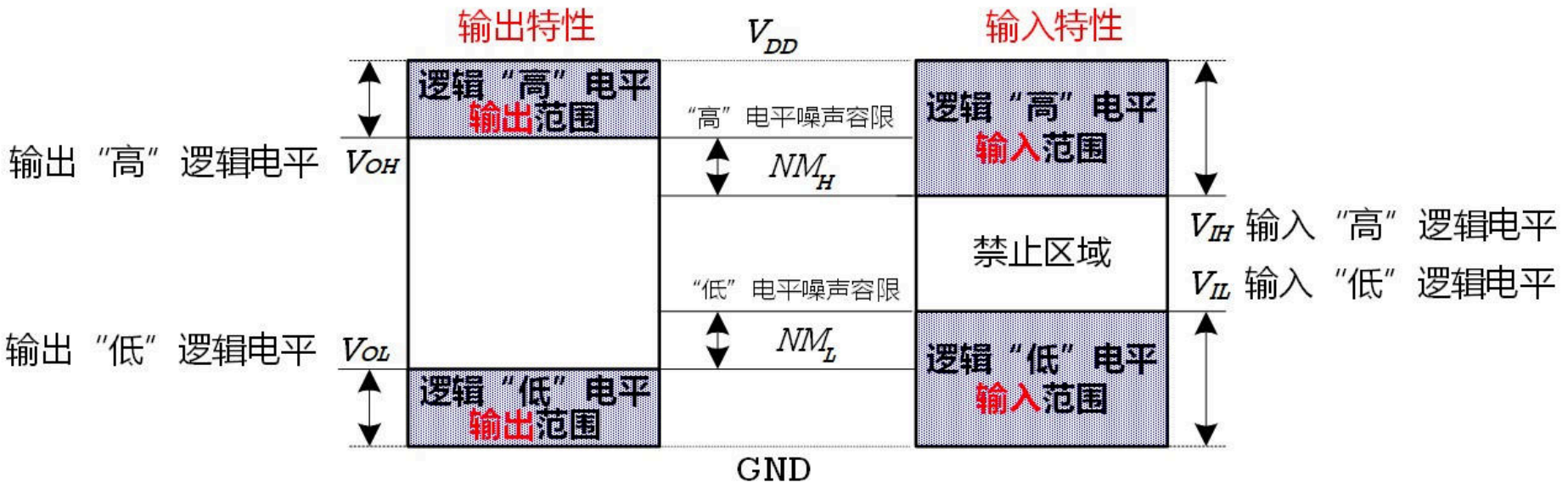
- 数字系统采用离散值变量（如 0v/GND 代表0, 5v/V_{DD} 代表1）
- 可现实中，电压是连续量
  - 那么4.99v代表什么？是变量0还是变量1?
  - 那么3.3v, 2.5v, 1.8v, 1.5v, 1.2v, 1.0v, … 都代表变量0还是变量1呢?

- **提出逻辑电平（Logic Levels）的概念**
  - 通过定义逻辑电平，将连续的变量映射到离散的二进制变量
  - 逻辑 1 与逻辑 0 对应的电压取值范围
  - 逻辑电路的输入、输出采用不同电压区间，以此容忍电路噪声的干扰

逻辑电平 Logic Levels



噪声容限（Noise Margins）

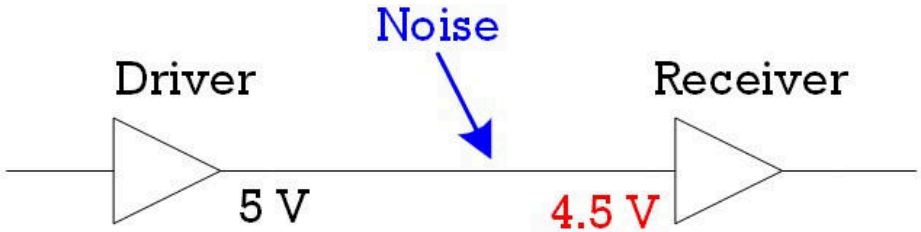


高电平噪声容限：$NM_H = V_{OH} - V_{IH}$

低电平噪声容限：$NM_L = V_{IL} - V_{OL}$

噪声来源

- 任何会造成信号衰减的因素
  - 例如：线路电阻、电源噪声、相邻导线信号串扰等。
- 示例：
  - 逻辑门（驱动端）输出电压为 5 伏，但长线缆存在电阻，接收端实际仅收到 4.5 伏。



静态约束

- 为了避免输入处于禁止区域，数字逻辑门的设计要遵循静态约束
  - 也就是说每个电路元件，对于给定的有效逻辑输入，它都要产生有效的逻辑输出。（电路元件都会给出这些指标）
- 使用有限范围内的电压来表示离散值

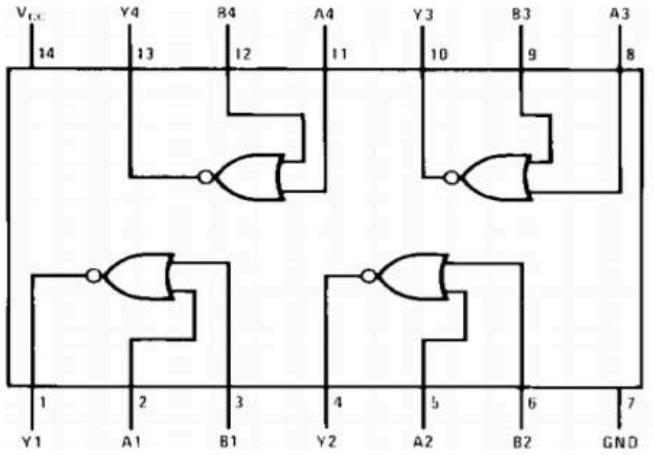
FAIRCHILD
SEMICONDUCTOR™

DM74LS02
Quad 2-Input NOR Gate

General Description

This device contains four independent gates each of which performs the logic NOR function.

Connection Diagram



Recommended Operating Conditions

Symbol	Parameter	Min	Nom	Max	Units
V_{CC}	Supply Voltage	4.75	5	5.25	V
V_{IH}	HIGH Level Input Voltage	2			V
V_{IL}	LOW Level Input Voltage			0.8	V
I_{OH}	HIGH Level Output Current			−0.4	mA
I_{OL}	LOW Level Output Current			8	mA
T_A	Free Air Operating Temperature	0		70	°C

Electrical Characteristics

over recommended operating free air temperature range (unless otherwise noted)

Symbol	Parameter	Conditions	Min	Typ (Note 2)	Max	Units
V_I	Input Clamp Voltage	V_{CC} = Min, I_I = −18 mA			−1.5	V
V_{OH}	HIGH Level Output Voltage	V_{CC} = Min, I_{OH} = Max, V_{IL} = Max	2.7	3.4		V
V_{OL}	LOW Level Output Voltage	V_{CC} = Min, I_{OL} = Max, V_{IH} = Min		0.35	0.5	V
		I_{OL} = 4 mA, V_{CC} = Min		0.25	0.4	
I_I	Input Current @ Max Input Voltage	V_{CC} = Max, V_I = 7V			0.1	mA
I_{IH}	HIGH Level Input Current	V_{CC} = Max, V_I = 2.7V			20	μA
I_{IL}	LOW Level Input Current	V_{CC} = Max, V_I = 0.4V			−0.40	mA
I_{OS}	Short Circuit Output Current	V_{CC} = Max (Note 3)	−20		−100	mA
I_{CCH}	Supply Current with Outputs HIGH	V_{CC} = Max		1.6	3.2	mA
I_{CCL}	Supply Current with Outputs LOW	V_{CC} = Max		2.8	5.4	mA

电源电压缩放 VDD Scaling

- 假设系统中最低的电压为0v，称为地(GND)
- 系统中最高的电压来自电源，常称为VDD
- 在1970-1980年代的技术下，一般VDD是 5V
- 当芯片采用了更小的晶体管，VDD下降了
  - 一方面避免微小的晶体管过热过载，另一方面省电
  - 所以VDD 3.3 V, 2.5 V, 1.8 V, 1.5 V, 1.2 V, 1.0 V, …
- 注意：切勿将不同供电电压的芯片直接相连
  - 电子圈经典工程师幽默段子：芯片之所以能够工作，是因为里面存有魔法烟雾。一旦魔法烟雾泄漏出来，芯片就失效了。



雨课堂 Rain Classroom

逻辑系列示例

- 连接不同供电电压的芯片时务必小心

Logic Family	V_{DD}	V_{IL}	V_{IH}	V_{OL}	V_{OH}
TTL	5 (4.75 - 5.25)	0.8	2.0	0.4	2.4
CMOS	5 (4.5 - 6)	1.35	3.15	0.33	3.84
LVTTL	3.3 (3 - 3.6)	0.8	2.0	0.4	2.4
LVCMOS	3.3 (3 - 3.6)	0.9	1.8	0.36	2.7

单选题 1分

逻辑电平（Logic Levels）被引入的根本目的是什么?

A 提高晶体管开关速度

B 将连续的模拟电压信号映射为离散的二进制逻辑值（0/1），从而实现数字抽象

C 降低CMOS电路的静态功耗

D 统一所有逻辑门的封装尺寸

雨课堂 Rain Classroom

Chapter 1 :: Topics

- 驾驭复杂的艺术 (The Art of Managing Complexity)
- 数制系统 (Number Systems)
- 数字抽象 (The Digital Abstraction)
- **逻辑门 (Logic Gates)**
- CMOS晶体管 (CMOS Transistors)
- 功耗 (Power Consumption)

雨课堂 Rain Classroom

布尔代数和逻辑表达式 （自学）

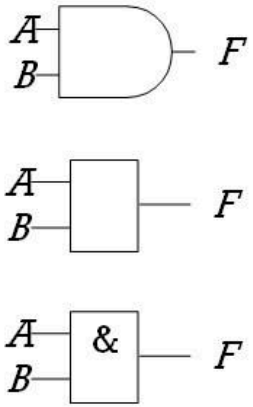
➢ AND Logic（Two-Input Logic Gates）

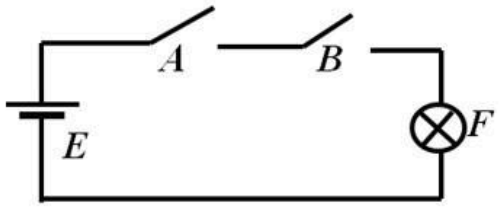
- F = A·B = AB = A∧B

truth table

A	B	F
0	0	0
0	1	0
1	0	0
1	1	1

AND





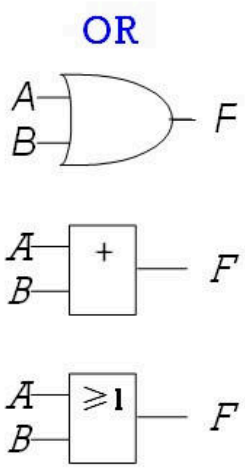
雨课堂 Rain Classroom

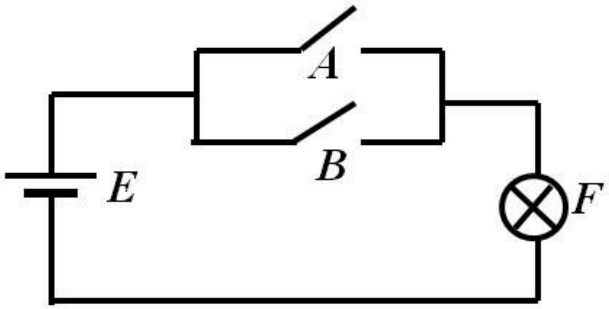
布尔代数和逻辑表达式 （自学）

- **OR Logic**（Two-Input Logic Gates）
  - F=A+B = A∨B

truth table

A	B	F
0	0	0
0	1	1
1	0	1
1	1	1





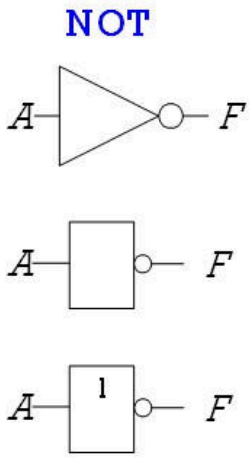
布尔代数和逻辑表达式 （自学）

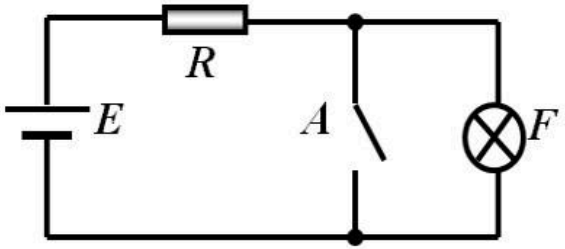
➢ NOT Logic（Single-Input Logic Gates）

- $F = \overline{A} = A' = \neg A$

truth table

A	F
0	1
1	0



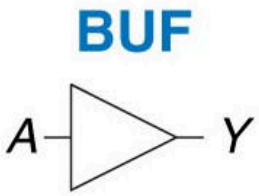


布尔代数和逻辑表达式 （自学）

- **缓冲门**(Buffer)（Single-Input Logic Gates）
  - F = A

truth table

A	*F*
0	0
1	1

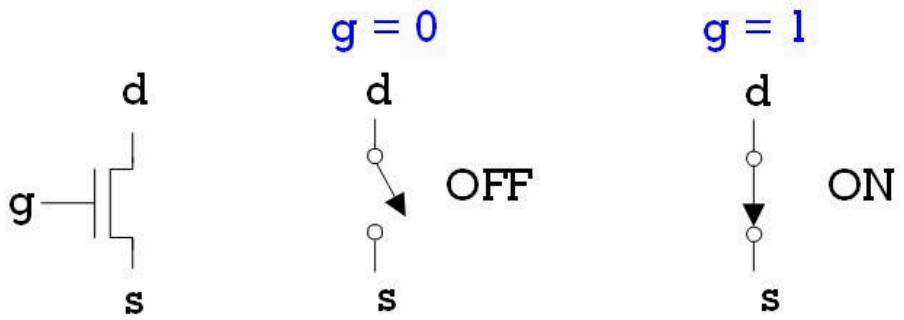


Chapter 1 :: Topics

- 驾驭复杂的艺术 (The Art of Managing Complexity)
- 数制系统 (Number Systems)
- 数字抽象 (The Digital Abstraction)
- 逻辑门 (Logic Gates)
- **CMOS晶体管 (CMOS Transistors)**
- 功耗 (Power Consumption)

晶体管 Transistors

- 逻辑门一般由晶体管组成
  - MOS晶体管是一个三端口的（栅极g、漏极d，源级s），电压控制的**电子开关**
  - 两个端口的通、断由第三个端口的电压决定
  - 当栅极 g 为逻辑 1 时，漏极 d 与源极 s 导通（开启状态）
  - BJT双极三极管也是一个三端口的（基极、发射极、集电极），电流控制的电子开关



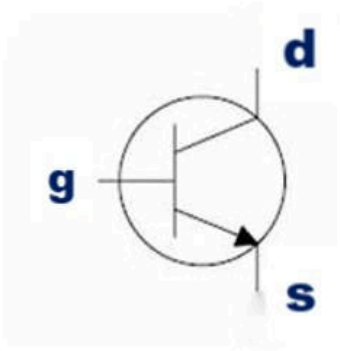
晶体管 Transistors

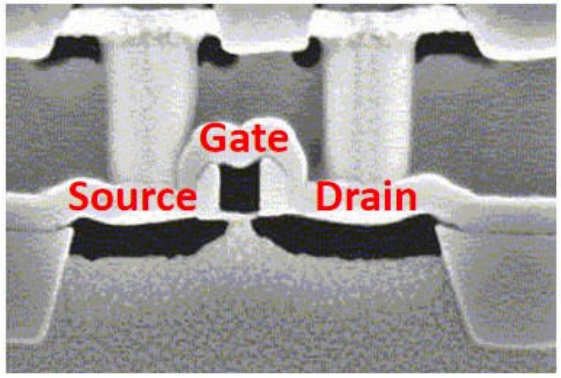
- 通过结合
  - 导体 (金属，Metal)
  - 绝缘体 (氧化，Oxide)



半导体，
Semiconductors

- 我们就得到了 (MOS晶体管)

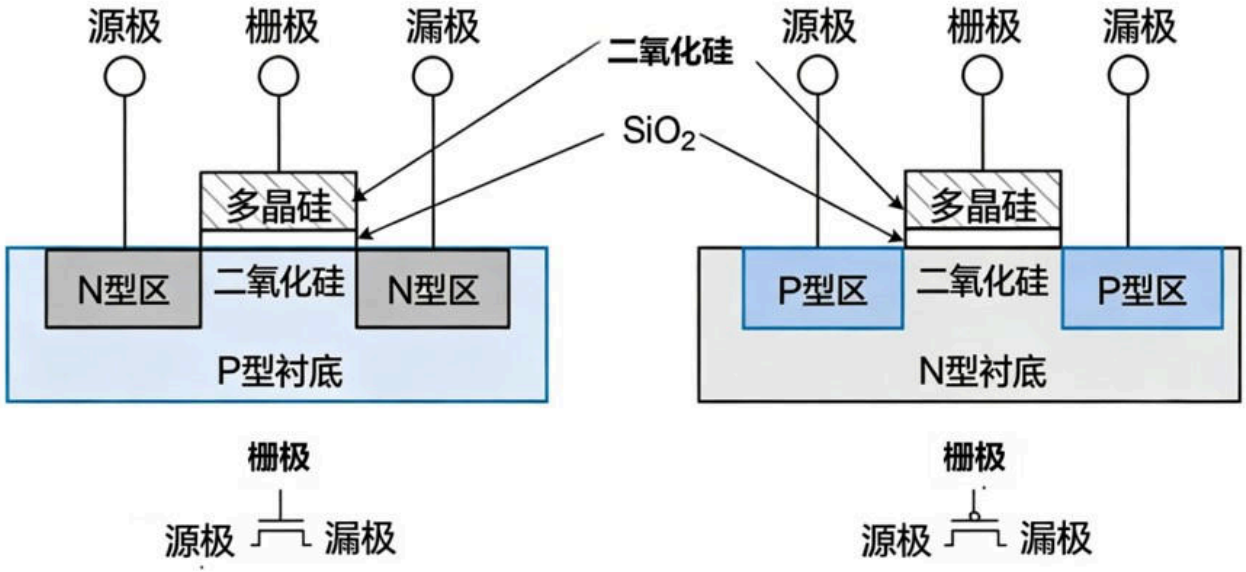






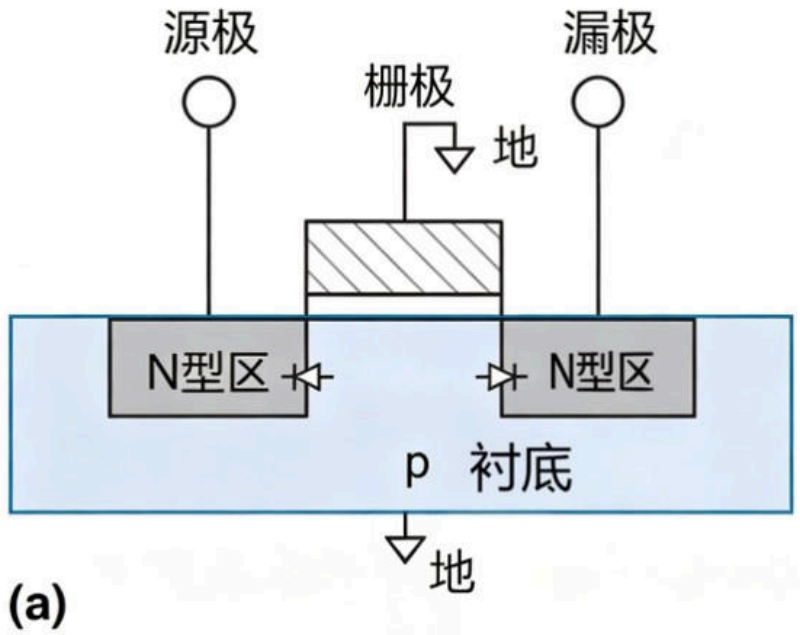
补充知识

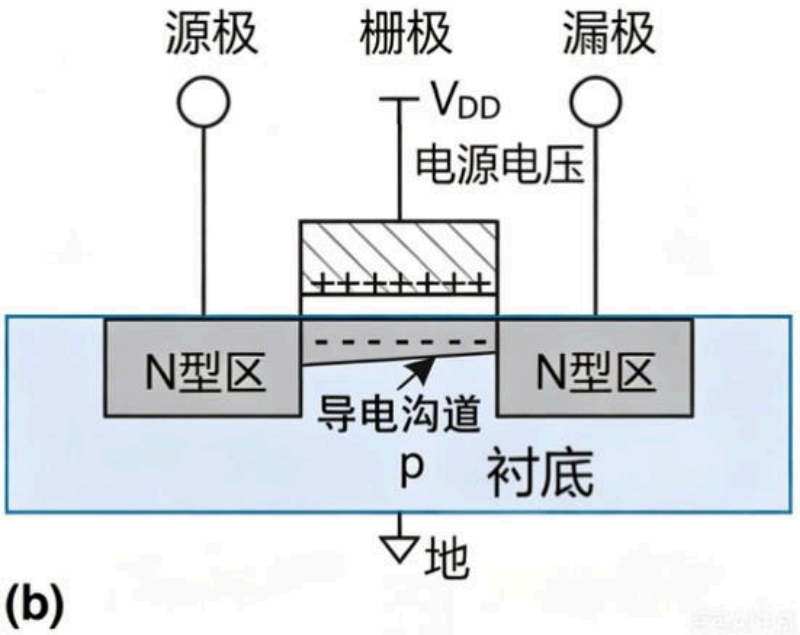
- 在数字电路中，MOS管工作在开关状态，即线性导通 & 截止



补充知识

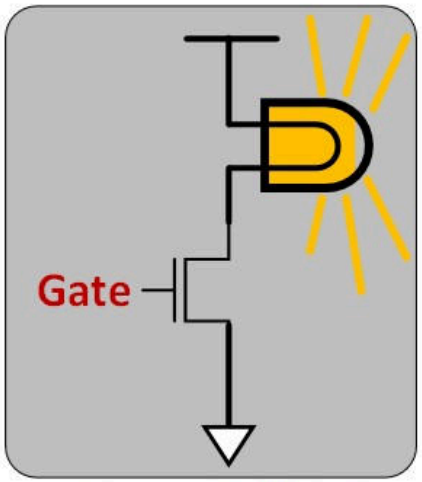
- 在数字电路中，MOS管工作在开关状态，即线性导通 & 截止



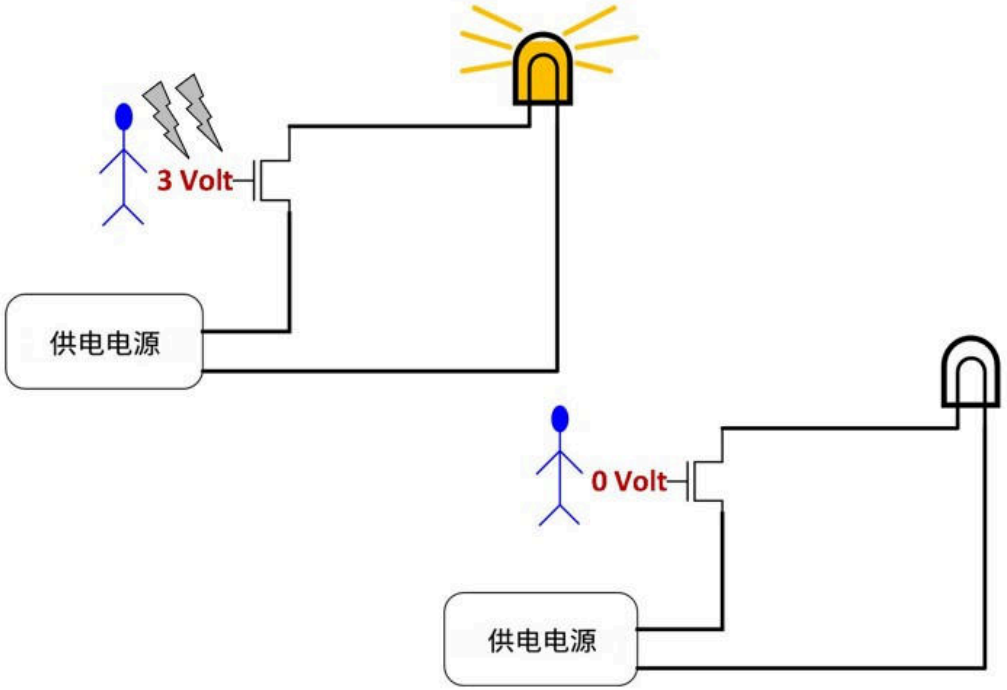


晶体管如何工作?

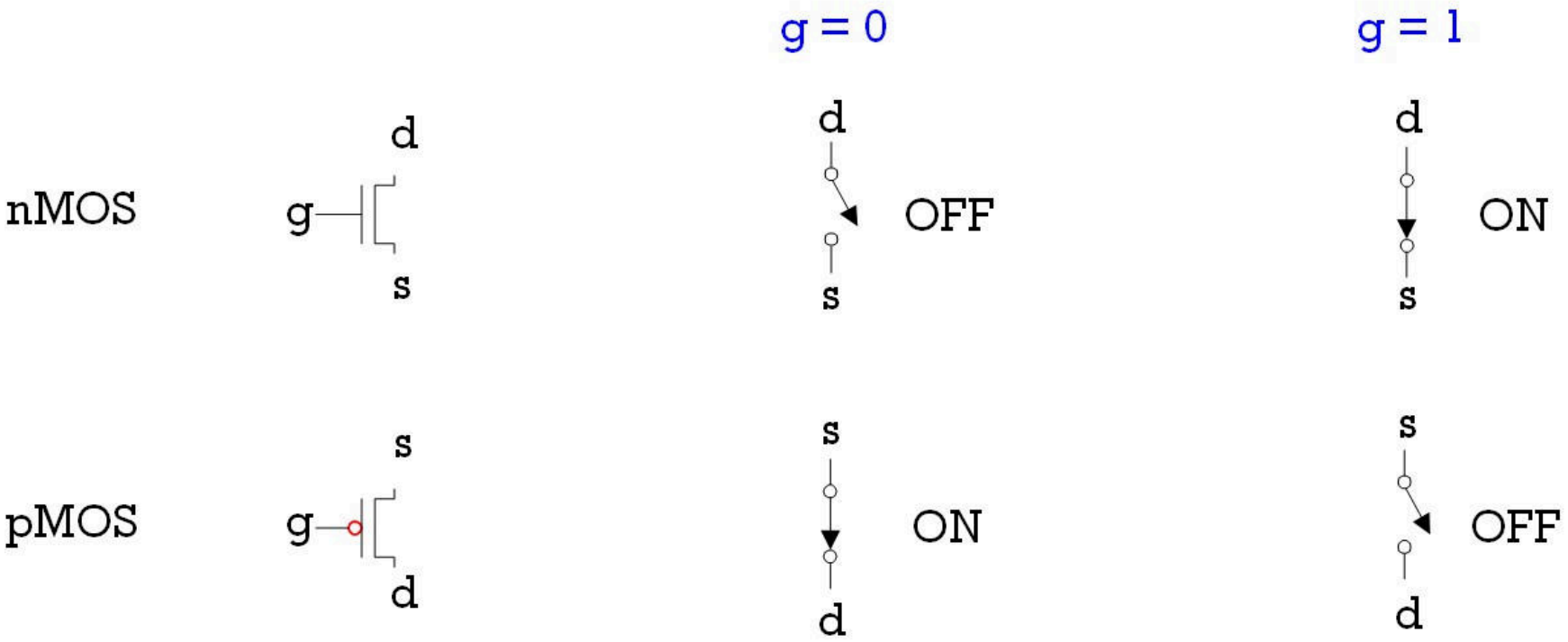
- N 型晶体管（NMOS 管）在带有电池与灯泡的电路中



右侧电路的简版图

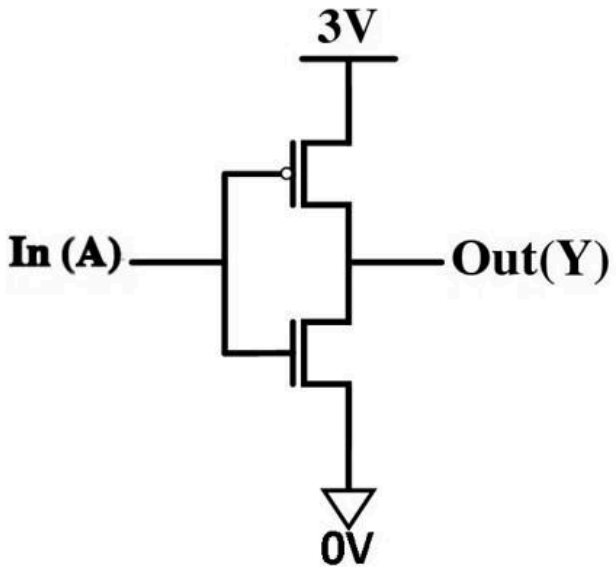


晶体管功能特性



采用 CMOS 工艺搭建逻辑单元

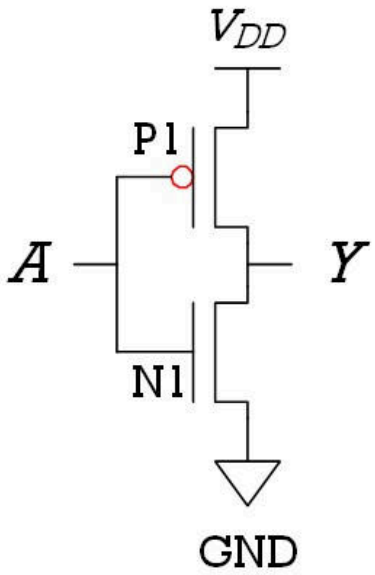
- 现代计算机同时采用 N 型与 P 型晶体管，也就是互补金属氧化物半导体工艺（CMOS，Complementary互补的 MOS technology）



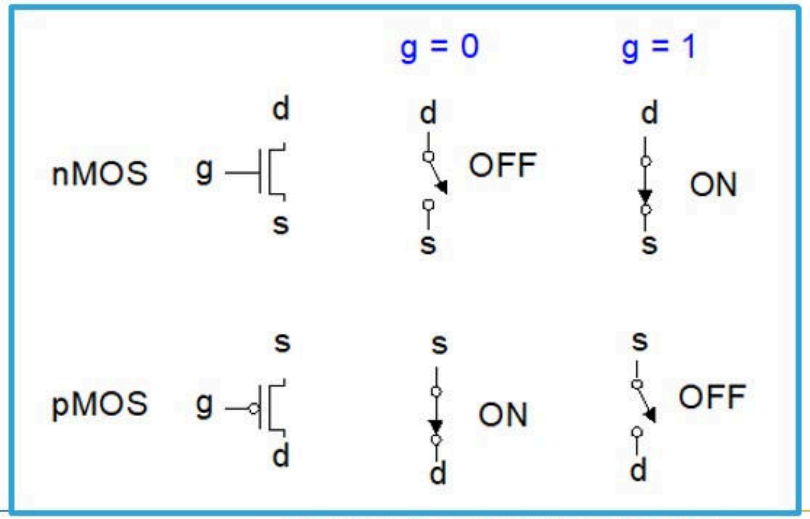
nMOS + pMOS = CMOS

这个电路什么功能?

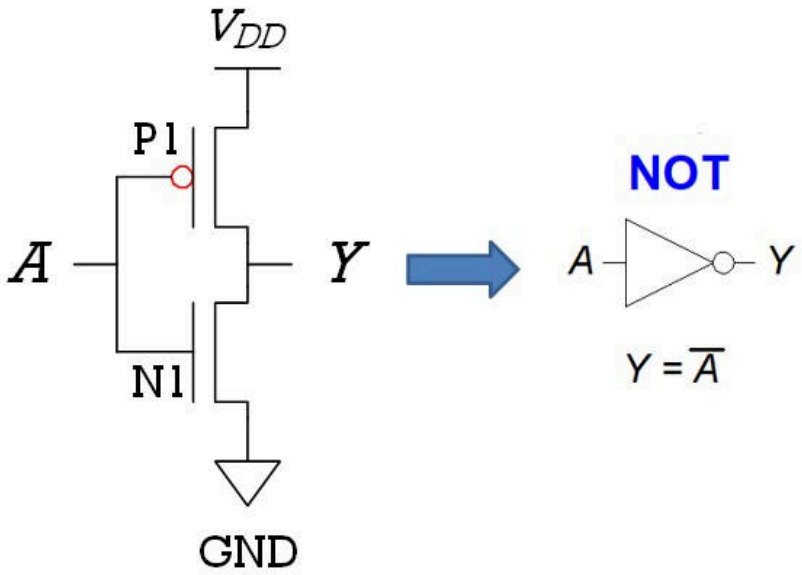
CMOS 逻辑门: NOT Gate



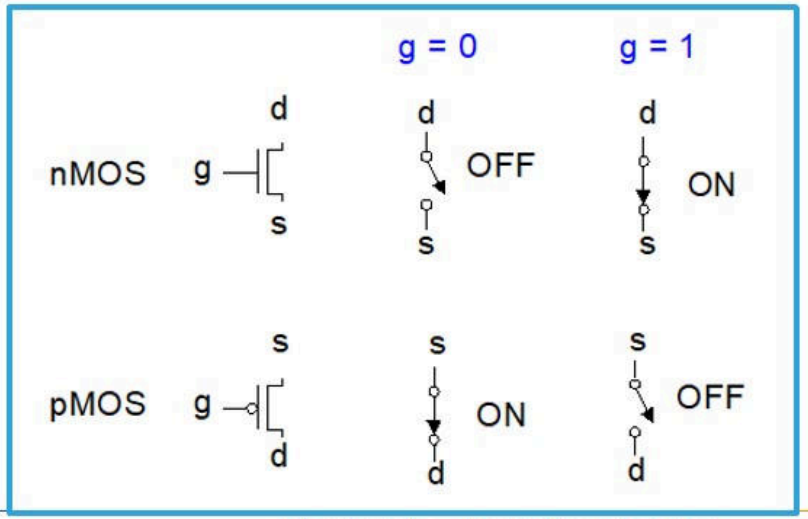
A	P1	N1	Y
0			
1			



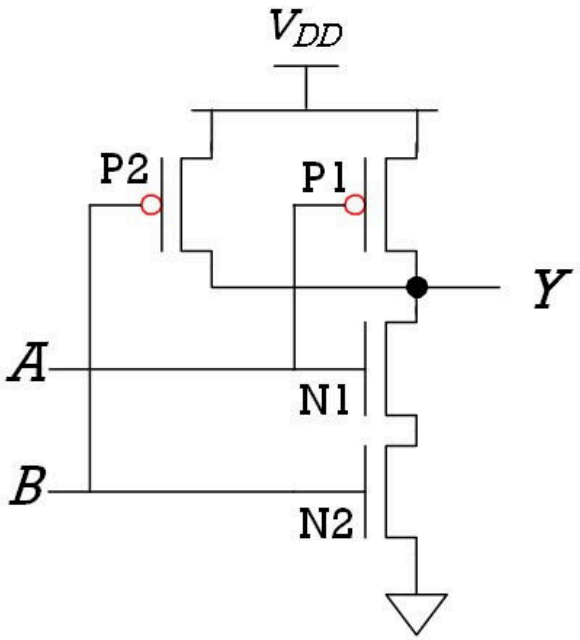
CMOS 逻辑门: NOT Gate

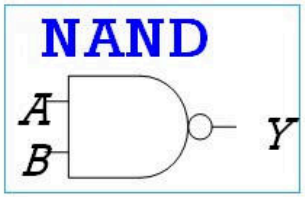


A	P1	N1	Y
0	ON	OFF	1
1	OFF	ON	0

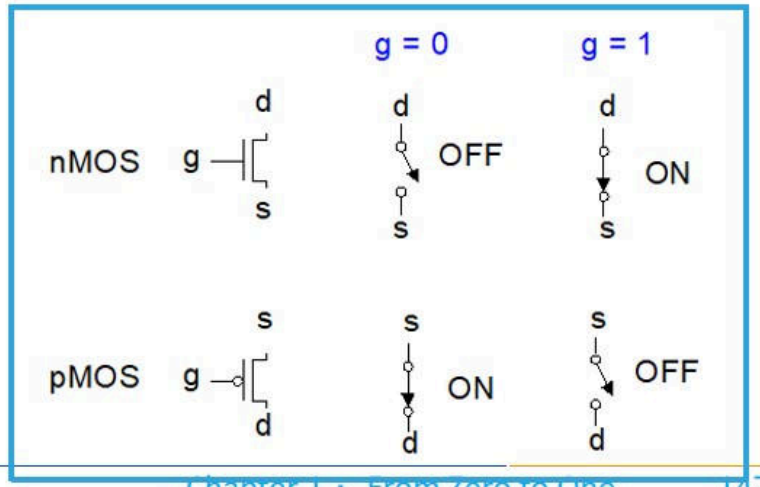


CMOS 逻辑门（慕课堂讨论）



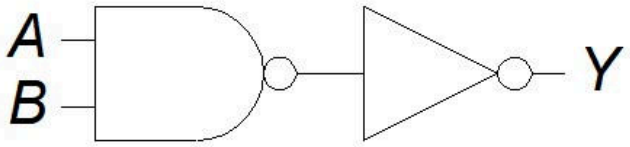


A	B	P1	P2	N1	N2	Y
0	0	ON	ON	OFF	OFF	1
0	1	ON	OFF	OFF	ON	1
1	0	OFF	ON	ON	OFF	1
1	1	OFF	OFF	ON	ON	0



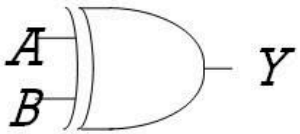
CMOS 逻辑门（慕课堂讨论）

- 如何构建一个两输入的 AND gate?



更多的两输入逻辑门

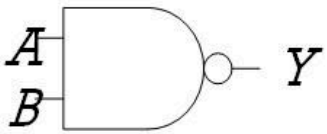
XOR



$Y = A \oplus B$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

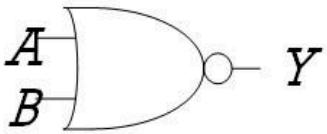
NAND



$Y = \overline{AB}$

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

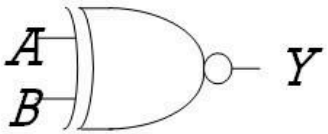
NOR



$Y = \overline{A + B}$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

XNOR



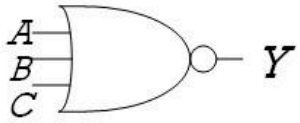
$Y = \overline{A \oplus B}$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

雨课堂 Rain Classroom

多输入逻辑门

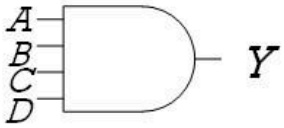
NOR 3



$$Y=\overline{A+B+C}$$

A	B	C	Y
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

AND 4



$$Y = ABCD$$

A	B	C	D	Y
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0

A	B	C	D	Y
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	1

雨课堂 Rain Classroom

Chapter 1 :: Topics

- 驾驭复杂的艺术 (The Art of Managing Complexity)
- 数制系统 (Number Systems)
- 数字抽象 (The Digital Abstraction)
- 逻辑门 (Logic Gates)
- CMOS晶体管 (CMOS Transistors)
- **功耗** **(Power Consumption)**

功耗 Power Consumption

- **功耗是单位时间内所消耗的能量**，在数字系统中非常重要。
  - 在手机、笔记本电脑等移动终端中，电池的使用时间取决于功耗。
  - 功耗对固定电源供电的设备也非常重要，因为电力消耗要花钱，而且功耗过高将导致系统过热。

- 数字系统包含
  - **动态功耗**（Dynamic power consumption）
  - **静态功耗**（Static power consumption）

动态功耗 Dynamic Power Consumption

- **动态功耗是信号在0和1之间变化，电容充电过程中所消耗的能量**
  - 将电容 C 充电至电压 V_{DD} 所需的能量为 CV_{DD}^2
  - 如果电路工作频率是 f，那么晶体管会以该频率完成电平翻转（1变0，或0变1）
  - 电容每秒完成 $f/2$ 次充电过程（电容从1放电至0的过程不消耗充电能量）

- **动态功耗的计算公式为：** $P_{dynamic} = \frac{1}{2} CV_{DD}^2 f$

- **功耗和电源电压 V_{DD} 的平方成正比，对电压极其敏感**
  - 电压减半，动态功耗直接变为原来的 1/4；

静态功耗 Static Power Consumption

- 静态功耗是逻辑门无电平翻转时产生的功耗
  - 由静态电源电流 I_{DD} (也叫漏电流) 引发
- 静态功耗的计算公式： $P_{static} = I_{DD} V_{DD}$
  - 漏电流 I_{DD} 本身也随 V_{DD} 增大而上升；
- 大家发现没有，无论动态还是静态功耗，都与 V_{DD} 有关
- 所以降低 V_{DD} 是降低芯片总功耗最有效的手段

雨课堂 Rain Classroom

功耗计算示例

- 估算一部手机的功耗
  - 假设该手机电池容量 6Wh（瓦小时）
  - 电源电压 $V_{DD} = 1.2$ V
  - 芯片中的平均电容 $C = 10$ nF
  - 手机通话时的工作频率 $f = 300$MHz
  - 天线需要3W功率
  - 手机不通话时动态功耗$P_{dynamic} = 0$，但仍然具有漏电流 $I_{DD} = 40$ mA

- **请计算不通话情况下和连续通话情况下，电池的使用时间**

雨课堂 Rain Classroom

功耗计算示例

- 假设该手机电池容量 6Wh（瓦小时），天线需要3W功率，电源电压 V_{DD} = 1.2 V
- 芯片中的平均电容 C = 10 nF，手机通话时的工作频率 f = 300MHz，
- 手机不通话时动态功耗 $P_{dynamic} = 0$，但仍然具有漏电流 I_{DD} = 40 mA

➢ **不通话情况下：**

- Pstatic = $I_{DD}V_{DD}$ = 40 mA×1.2 V = 48mW
- 所以电池使用时间：6Wh/48mW = 125小时 ≈ 5天

➢ **连续通话情况下：**

- P = 动态功耗 + 静态功耗 +天线功耗 = $\frac{1}{2}CV_{DD}^2f + I_{DD}V_{DD} + 3W$

  = ½ (10nF)(1.2V)²(300MHz) + (40mA)(1.2V) + 3W = **5.208W**

- 所以电池使用时间：6Wh/5.208W = 1.15小时

雨课堂 Rain Classroom

第三堂课结束了

有问题同学们可以提问讨论

雨课堂 Rain Classroom